

Low-Level Design Document : CBE Super App - BPS Action Service

1. Document Information

Attribute	Description
Document Title	Low-Level Design for BPS Action Service
Author(s)	<u>Yishak Terefe</u>
Reviewed By	Yohannes Teshome & Yeabsira Gashaw
Version	1.0
Date	2025-07-24

Table of Contents

1. [Executive Summary](#)
2. [Service Overview](#)
3. [Architecture](#)
4. [Data Models](#)
5. [BPS Actions - Detailed Implementation](#)
6. [API Specifications](#)
7. [Security & Authentication](#)
8. [Error Handling](#)



9. [Performance & Scalability](#)

10. [Monitoring & Observability](#)

1. Executive Summary

1.1 Purpose

The **BPS (Branch Processing System) Action Service** is a critical microservice in the CBE Super App ecosystem that implements the **Maker-Checker** dual authorization pattern for high-risk banking operations. This document provides comprehensive architectural and implementation details for all supported actions.

1.2 Key Capabilities

- **30+ Action Types:** Account management, security operations, profile updates, transaction limits
- **Maker-Checker Workflow:** Dual authorization for fraud prevention
- **Audit Trail:** Complete history of all sensitive operations
- **OTP Integration:** Multi-factor authentication for critical actions
- **Real-time Notifications:** Kafka-based event streaming

1.3 Technology Stack

Component	Technology	Version
Runtime	Go	1.25.2
Web Framework	Chi	5.2.3
Database	MongoDB	2.2.1
Cache	Redis	8.11.5
Message Queue	Kafka (Sarama)	1.46.0

2. Service Overview

2.1 Business Context

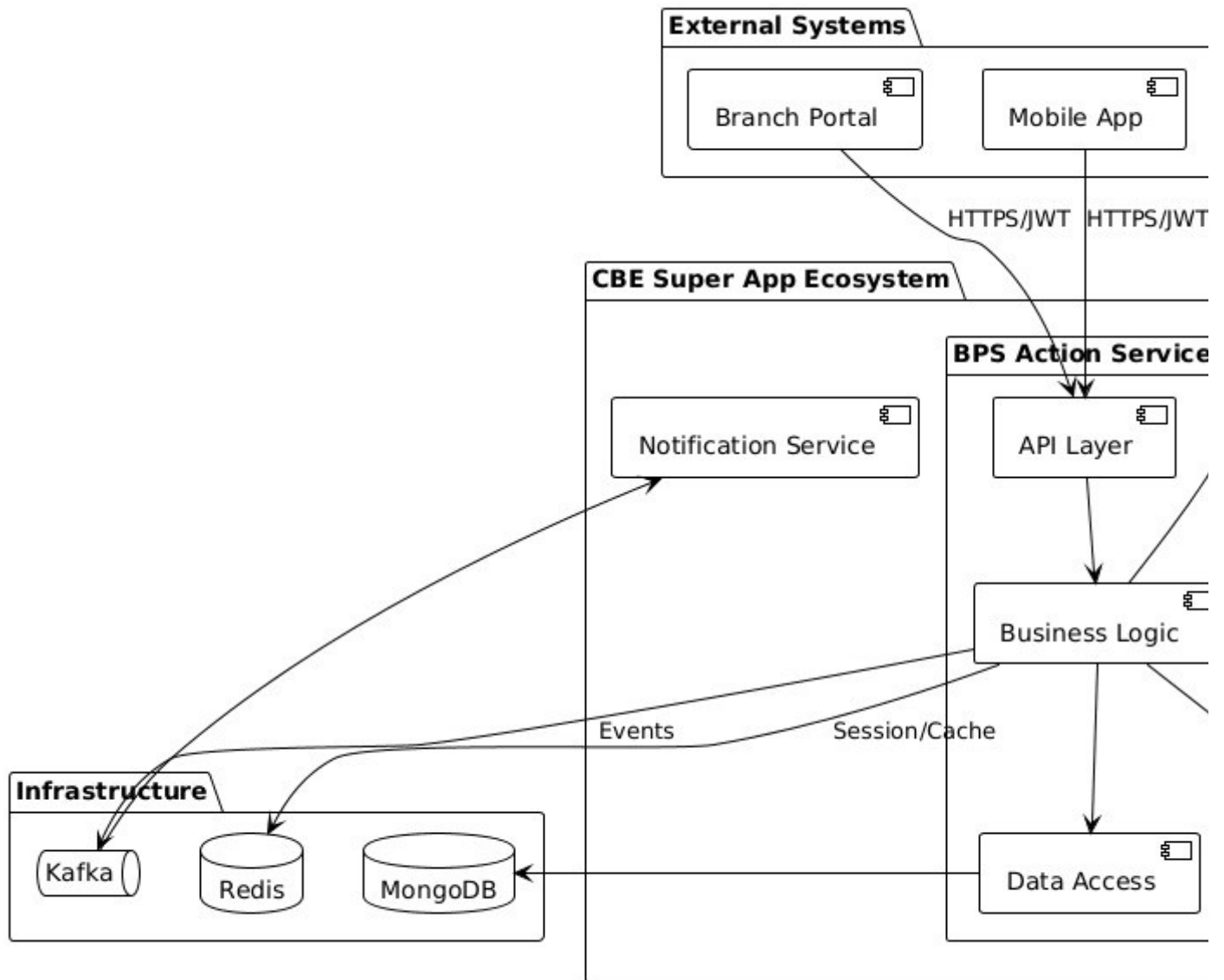
The BPS Action Service implements the **Four-Eyes Principle** (Maker-Checker pattern), a fundamental control in banking systems where:

- **Maker:** Branch staff who initiates a sensitive operation
- **Checker:** Different branch staff who reviews and approves/rejects

This dual authorization prevents:

- Fraudulent transactions
- Unauthorized account modifications
- Data entry errors
- Insider threats

2.2 Service Boundaries



2.3 Supported Action Types

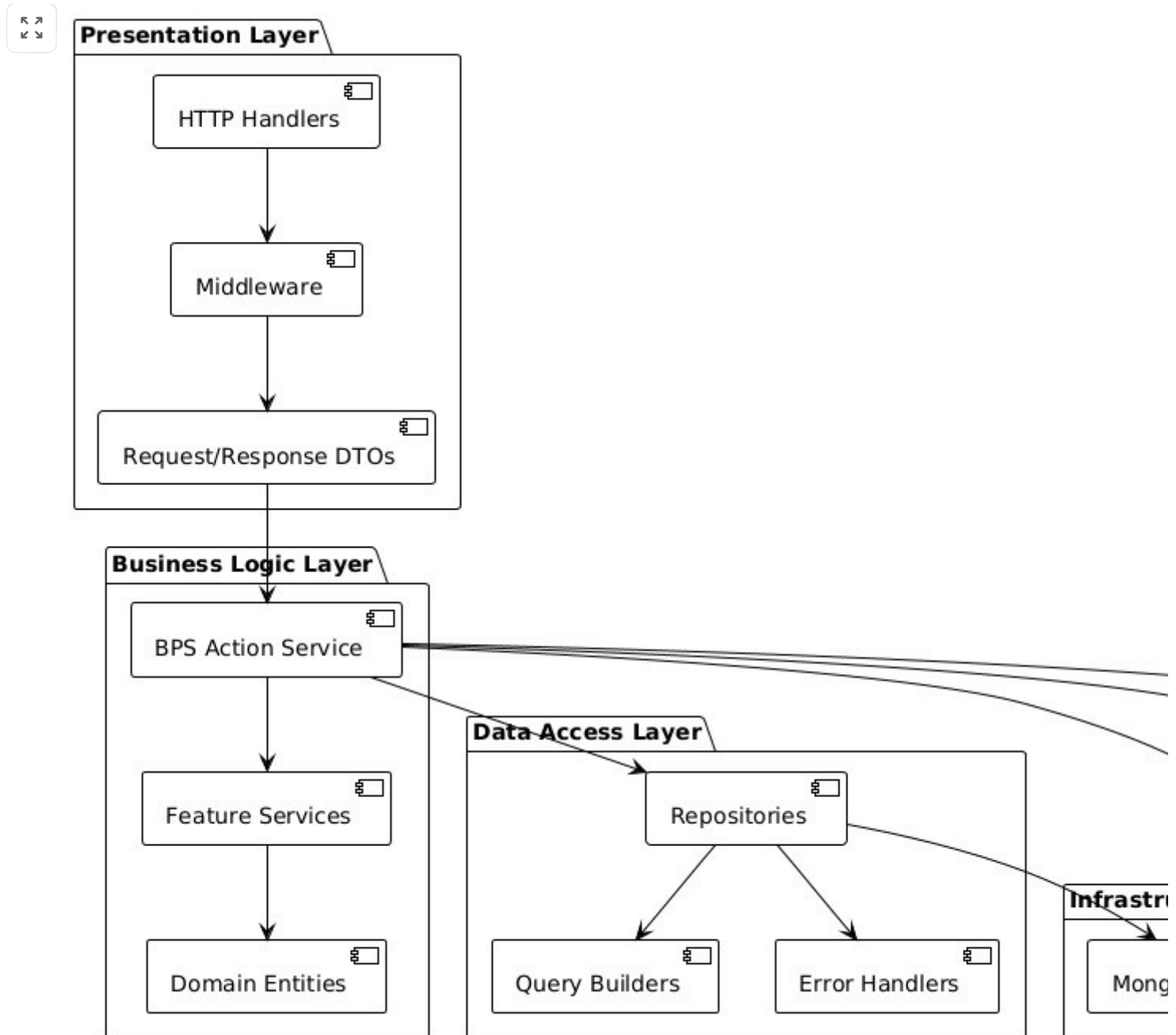
Category	Actions	Count
Account Management	LINK_ACCOUNT, UNLINK_ACCOUNT, ADD_ACCOUNT, LINK_ANDOR_ACCOUNT	4
Security Operations	UNLINK_DEVICE, ENABLE_BLOCKED, DISABLE_BLOCKED	4
Profile Management	CHANGE_PHONE_NUMBER, CHANGE_EMAIL, ATTACH_PHONE_NUMBER, DETACH_PHONE_NUMBER	4

User Status	ENABLE_USER, DISABLE_USER, ENABLE_DISABLE_USER, ACTIVATE_ACCOUNT, REACTIVATE	5
Transaction Limits	UPDATE_TRANSFER_LIMIT, CREATE_TRANSFER_LIMIT, RESET_TRANSFER_LIMIT, UPDATE_ONE_LIMIT, UPDATE_LIMIT, LIMIT_TRANSFER	6
Access Control	ACCESS_CONTROL, RESET_ACCESS_CONTROL	2
KYC	UPGRADE_KYC_LEVEL	1
Cheque	CREATE_CHEQUE_AUTHORIZATION_ACTION	1
Account Actions	REJECT_ACCOUNT, UPGRADE_ACCOUNT	2

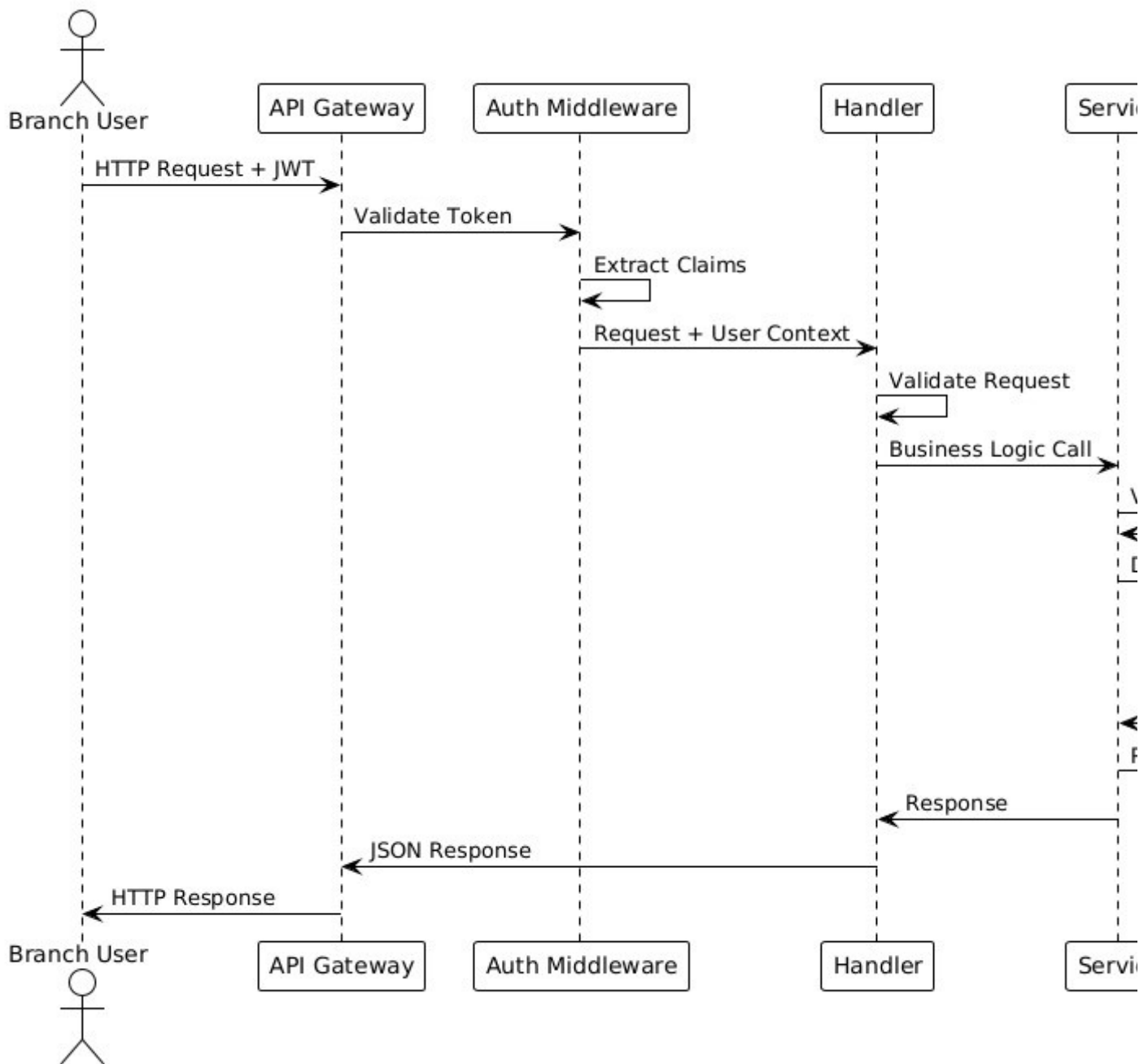
Total: 30 Action Types

3. Architecture

3.1 Clean Architecture Layers



3.2 Component Interaction Flow



3.3 Directory Structure

cbe-super-app-bps-action/

```

├── cmd/
│   └── main.go           # Application entry point
├── config/
│   ├── env.go           # Configuration loader
│   ├── mongo.go         # MongoDB setup
│   └── redis.go          # Redis setup
├── internal/
│   ├── constants/
│   │   ├── dto/         # Request/Response DTOs
│   │   └── entities/    # Domain models

```

```

├── errors/           # Error definitions
├── interfaces/       # Port interfaces
├── handlers/
│   ├── middleware/   # HTTP middleware
│   └── rest/         # REST handlers
├── service/         # Business logic
│   ├── bps_action/   # Core action service
│   ├── linked_account/ # Account linking
│   ├── member/       # User management
│   ├── otp/          # OTP service
│   └── transfer_limit/ # Limit management
├── storage/         # Data access
│   ├── bps_action/   # Action repository
│   ├── member/       # Member repository
│   └── otp/          # OTP repository
├── kafka/           # Kafka integration
├── docs/            # Documentation
└── scripts/         # Deployment scripts

```

4. Data Models

4.1 Core Entity: BPSAction

The BPSAction entity is the central data model that tracks all maker-checker operations.

MongoDB Collection: bps_actions

```

type BPSAction struct {
    ID          string          `bson:"_id" json:"id"`
    ActionCode  string          `bson:"action_code" json:"action_code"`

    // User Information
    UserInformation struct {
        UserID    string `bson:"user_id" json:"user_id"`
        UserCode  string `bson:"user_code" json:"user_code"`
        FullName  string `bson:"full_name" json:"full_name"`
    } `bson:"user" json:"user_information"`

    // Business Information (for merchant actions)
    BusinessInformation struct {
        BusinessID string `bson:"business_id" json:"business_id"`
        TILLNumber  string `bson:"till_number" json:"till_number"`
    }
}

```



```

    BusinessName string `bson:"business_name" json:"business_name"`
} `bson:"business" json:"business_information"`

// Maker Information
MakerID          string `bson:"maker_user" json:"maker_id"`
MakerName        string `bson:"maker_name" json:"maker_name"`
MakerPhoneNumber string `bson:"maker_phone_number" json:"maker_phone_number"`

// Checker Information
CheckerID          string `bson:"checker_user" json:"checker_id"`
CheckerName        string `bson:"chceker_name" json:"checker_name"` // Note:
typo in DB
CheckerPhoneNumber string `bson:"checker_phone_number"
json:"checker_phone_number"`

// Action Details
ActionReason struct {
    ActionType string `bson:"action_type" json:"action_type"` // create,
update, delete
    ActionNote string `bson:"action_note" json:"action_note"`
} `bson:"action_reason" json:"action_reason"`

RequestAction string `bson:"request_action"
json:"request_action"`
EntityIdentifier string `bson:"entity_identifier"
json:"entity_identifier"`

// Location Information
HomeBranch string `bson:"home_branch" json:"home_branch"`
AccountBranchCode string `bson:"account_branch_code"
json:"account_branch_code"`
DistrictCode string `bson:"district_code" json:"district_code"`
BranchCode string `bson:"branch_code" json:"branch_code"`
LinkedDistrictCode string `bson:"linked_district_code"
json:"linked_district_code"`

// Account Information
AccountNumber string `bson:"account_number" json:"account_number"`
AccountHolderName string `bson:"account_holder_name"
json:"account_holder_name"`
ServiceName string `bson:"service_name" json:"service_name"`

// Change Tracking

```

```
CurrentAction  map[string]interface{} `bson:"current_action"
json:"current_action"`
PreviousAction map[string]interface{} `bson:"previous_action"
json:"previous_action"`

// Status & Timestamps
Status        string `bson:"status" json:"status"` // PENDING, APPROVED,
REJECTED
VerifiedAt    string `bson:"verified_at" json:"verified_at"`
CreatedAt     string `bson:"created_at" json:"created_at"`
LastModifiedAt string `bson:"last_modified_at" json:"last_modified_at"`
}
```

Field Descriptions:

Field	Type	Description	Required
action_code	string	Unique identifier (e.g., "ACT-2024-001")	Yes
request_action	string	Type of action (e.g., "LINK_ACCOUNT")	Yes
status	string	PENDING, APPROVED, REJECTED	Yes
maker_user	string	ID of the user who initiated	Yes
checker_user	string	ID of the user who approved/rejected	No
current_action	map	New state/data after action	Yes
previous_action	map	Old state/data before action	No
entity_identifier	string	User code or service key	Yes

4.2 Supporting Entities

User/Member Entity

```

type User struct {
    ID            string    `bson:"_id"`
    UserCode      string    `bson:"user_code"`
    FullName      string    `bson:"full_name"`
    PhoneNumber   string    `bson:"phone_number"`
    Email         string    `bson:"email"`
    BranchCode    string    `bson:"branch_code"`
    AccountStatus string    `bson:"account_status" // ACTIVE, INACTIVE
    BPSStatus     string    `bson:"bps_status"     // AUTHORIZED, DENIED,
PENDING
    KYCLevel      uint8     `bson:"kyc_level"`
    Enabled       bool      `bson:"enabled"`
    IsBlocked     bool      `bson:"is_blocked"`
    LoginPIN      LoginPIN  `bson:"login_pin"`
    CreatedAt     time.Time `bson:"created_at"`
    LastModifiedAt time.Time `bson:"last_modified_at"`
}

```

OTP Entity

```

type OTP struct {
    ID            string    `bson:"_id"`
    PhoneNumber   string    `bson:"phone_number"`
    UserCode      string    `bson:"user_code"`
    OTPCode       string    `bson:"otp_code"`
    OTPFor        string    `bson:"otp_for" // LOGIN, RESET_PIN, LINK_ACCOUNT,
etc.
    Status        string    `bson:"status" // PENDING, VERIFIED, DENIED
    ExpiresAt     time.Time `bson:"expires_at"`
    CreatedAt     time.Time `bson:"created_at"`
}

```

LinkedAccount Entity

```

type LinkedAccount struct {
    ID            string    `bson:"_id"`
    UserID        string    `bson:"user_id"`
    AccountNumber string    `bson:"account_number"`
    AccountHolderName string `bson:"account_holder_name"`
    CustomerNumber string    `bson:"customer_number"`
    LinkedStatus  bool      `bson:"linked_status"`
    IsMain        bool      `bson:"is_main"`
}

```

```
LinkedAt      time.Time `bson:"linked_at"`  
CreatedAt     time.Time `bson:"created_at"`  
}
```

5. BPS Actions - Detailed Implementation

This section provides comprehensive architectural implementation details for each BPS action type, including sequence diagrams, business rules, and code flows.

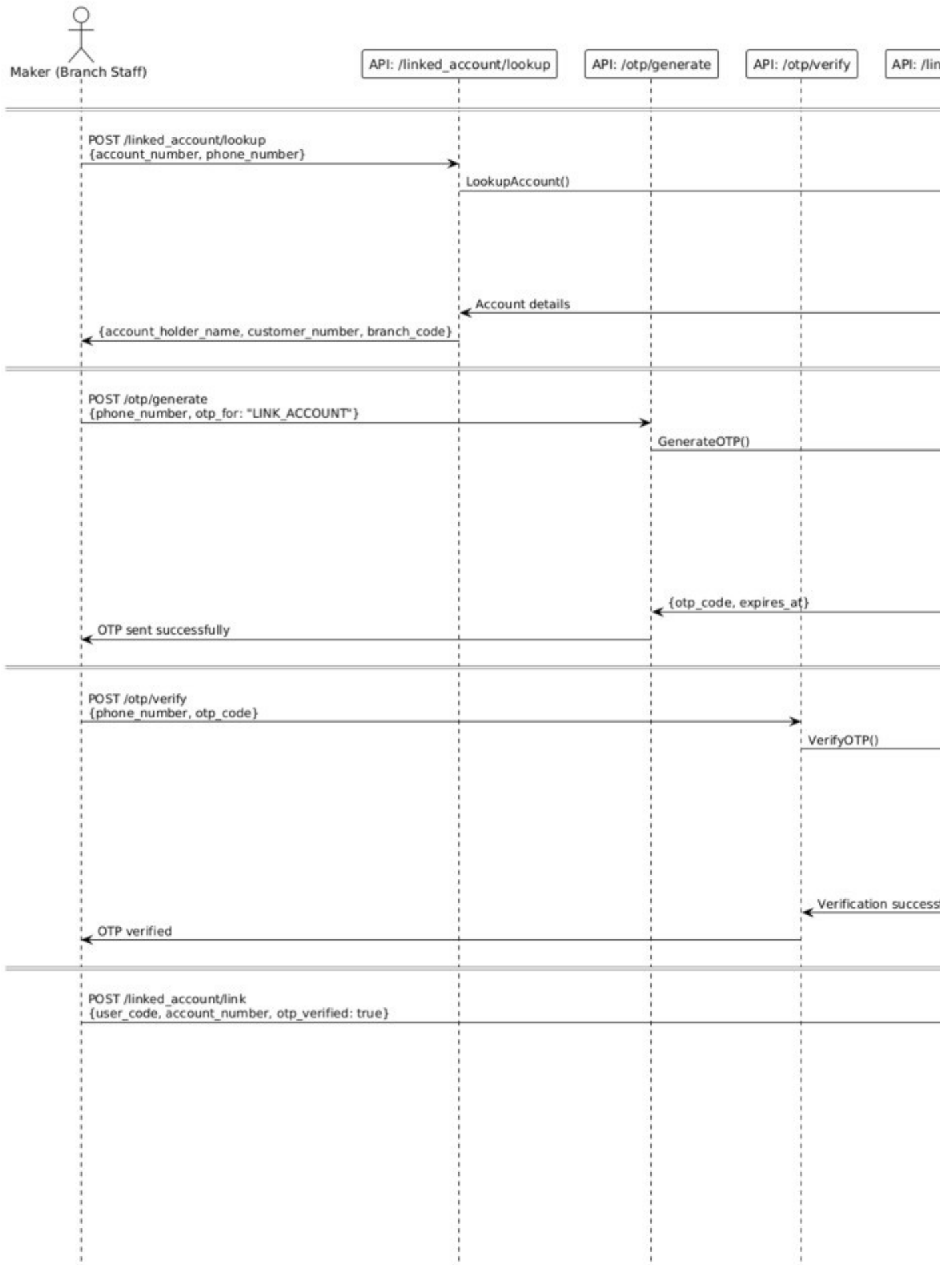
5.1 LINK_ACCOUNT

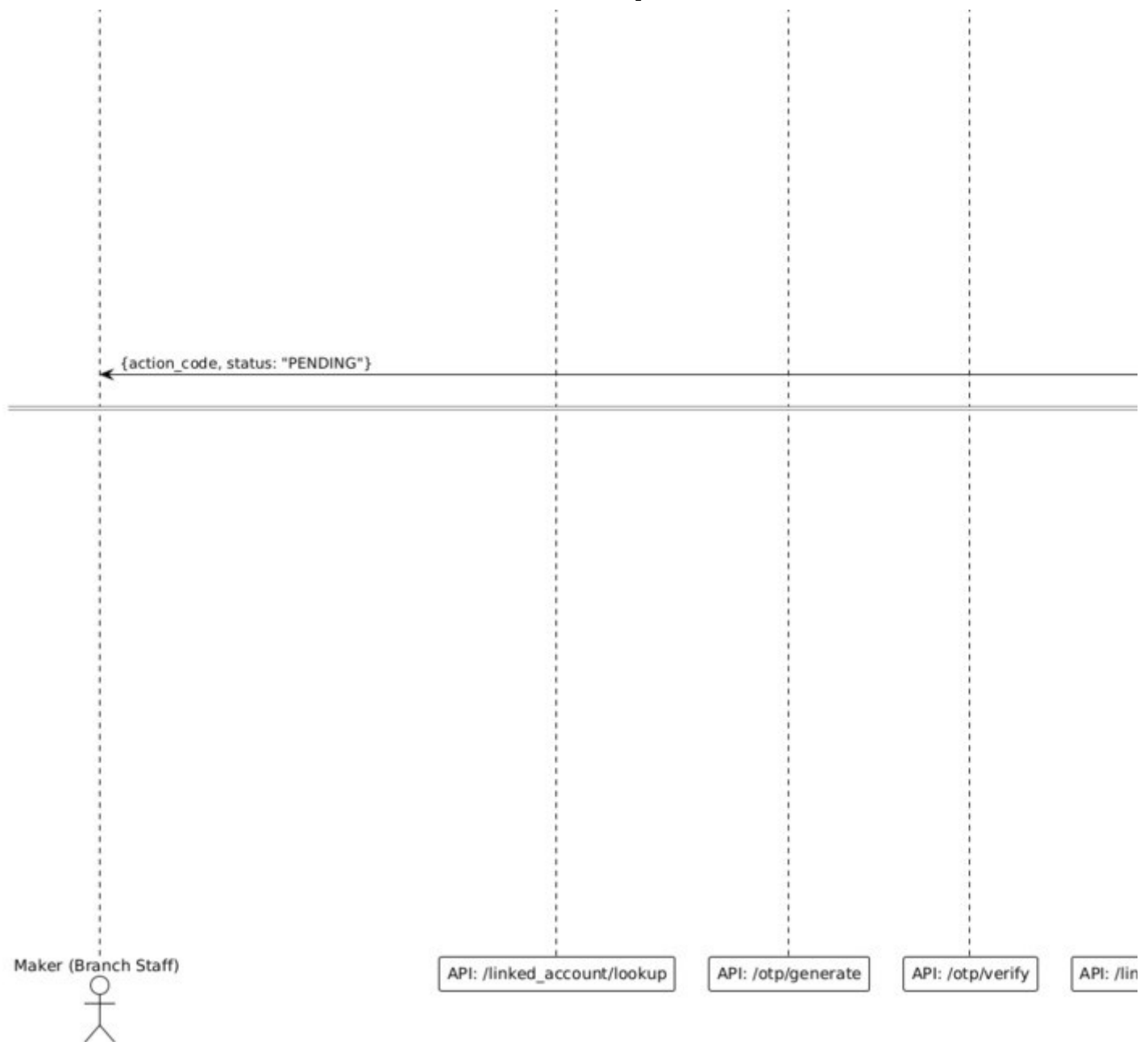
Purpose: Link a bank account to a user's mobile app profile.

Business Rules:

1. Account must exist in Core Banking System (CBS)
2. Account holder name must match user's registered name
3. User can have multiple linked accounts but only one main account
4. OTP verification required before creating BPS action
5. Maker cannot approve their own action
6. Account cannot be linked if already linked to another user

Architectural Flow:





Implementation Details:

1. Account Lookup Handler (internal/handlers/rest/linked_account/lookup.go)

```

func (h *Handler) LookupAccount(w http.ResponseWriter, r *http.Request) {
    var req dto.AccountLookupRequest
    json.NewDecoder(r.Body).Decode(&req)

    // Validate request
    if err := req.Validate(); err != nil {
        respondWithError(w, 400, err.Error())
        return
    }
}

```

```

// Call service
accountData, err := h.service.LookupAccount(r.Context(), req.AccountNumber,
req.PhoneNumber)
if err != nil {
    handleServiceError(w, err)
    return
}

respondWithSuccess(w, 200, "Account found", accountData)
}

```

2. Link Account Service (internal/service/linked_account/service.go)

```

func (s *Service) LinkAccount(ctx context.Context, userCode, accountNumber string,
maker entities.BPSUser) (entities.BPSAction, error) {
    // 1. Fetch user
    user, err := s.memberService.FetchUserByCode(ctx, userCode)
    if err != nil {
        return entities.BPSAction{}, err
    }

    // 2. Check for pending actions
    pendingActions, err := s.bpsActionService.CheckPendingActions(ctx,
maker.BranchCode, userCode, "LINK_ACCOUNT")
    if len(pendingActions) > 0 {
        return entities.BPSAction{}, errors.NewAppError("BPS-409", "Pending action
exists")
    }

    // 3. Prepare action data
    currentAction := map[string]interface{}{
        "account_number": accountNumber,
        "action_type": "link",
    }

    // 4. Create BPS Action
    action, err := s.bpsActionService.BpsActionCreate(
        ctx,
        maker,
        enums.ActionTypeCreate,
        "Link account request",
        userCode,
        user,
    )
}

```

```
        nil, // no previous state
        currentAction,
    )

    return action, err
}
```

3. BPS Action Approval (internal/service/bps_action/bps_action_service.go)

```
func (s *Store) BpsActionApprove(ctx context.Context, actionCode, actionNote
string, checker entities.BPSUser) (entities.BPSAction, *errors.AppError) {
    // Fetch action
    action, err := s.FetchBpsAction(ctx, actionCode)
    if err != nil {
        return entities.BPSAction{}, err
    }

    // Business rule: Maker ≠ Checker
    if action.MakerID == checker.ID {
        return entities.BPSAction{}, errors.NewAppError(
            "BPS-403",
            "Checker cannot approve their own action",
            errors.CategoryBusiness,
            errors.SeverityMedium,
        )
    }

    // Business rule: Only PENDING can be approved
    if action.Status != "PENDING" {
        return entities.BPSAction{}, errors.NewAppError(
            "BPS-409",
            "Action is not in PENDING status",
            errors.CategoryBusiness,
            errors.SeverityMedium,
        )
    }

    // Update action
    action.Status = "APPROVED"
    action.CheckerID = checker.ID
    action.CheckerName = checker.FullName
    action.VerifiedAt = time.Now().Format(time.RFC3339)
```



```
// Save to database
updatedAction, err := s.storage.UpdateAction(ctx, action)

// Execute business logic based on request_action
switch action.RequestAction {
case "LINK_ACCOUNT":
    s.executeLinkAccount(ctx, action)
case "RESET_PIN":
    s.executeResetPIN(ctx, action)
// ... other actions
}

return updatedAction, nil
}
```

Database Changes:

1. **bps_actions collection**: New document with status=PENDING
2. **linked_account collection** (after approval): New linked account record
3. **members collection** (after approval): Update account_linked flag

API Endpoints:

- POST /api/v1/cbesuperapp/bps_action/linked_account/lookup
- POST /api/v1/cbesuperapp/bps_action/otp/generate
- POST /api/v1/cbesuperapp/bps_action/otp/verify
- POST /api/v1/cbesuperapp/bps_action/linked_account/link
- POST /api/v1/cbesuperapp/bps_action/bps_action/approve

5.3 CHANGE_PHONE_NUMBER

Purpose: Update a user's registered phone number.

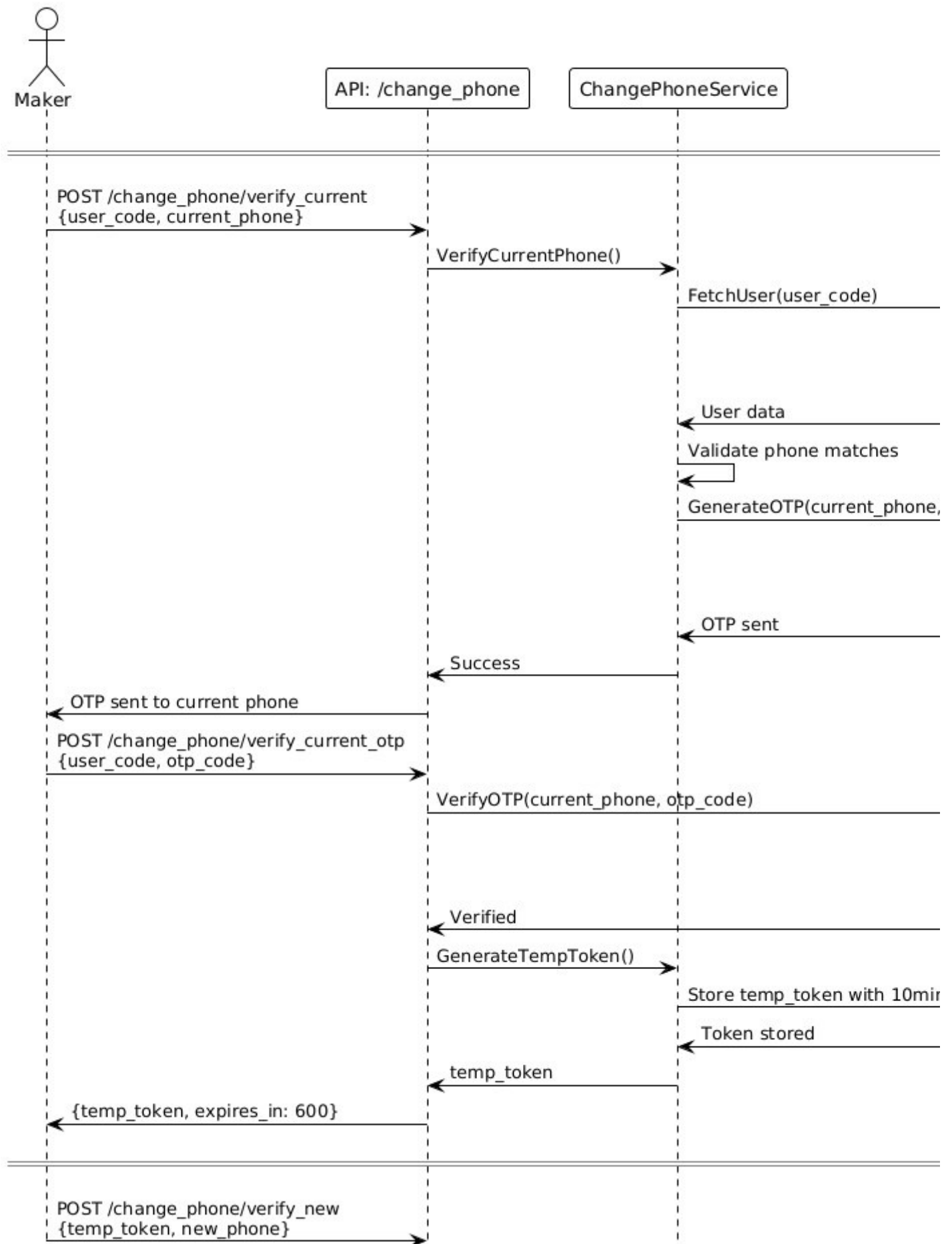
Business Rules:

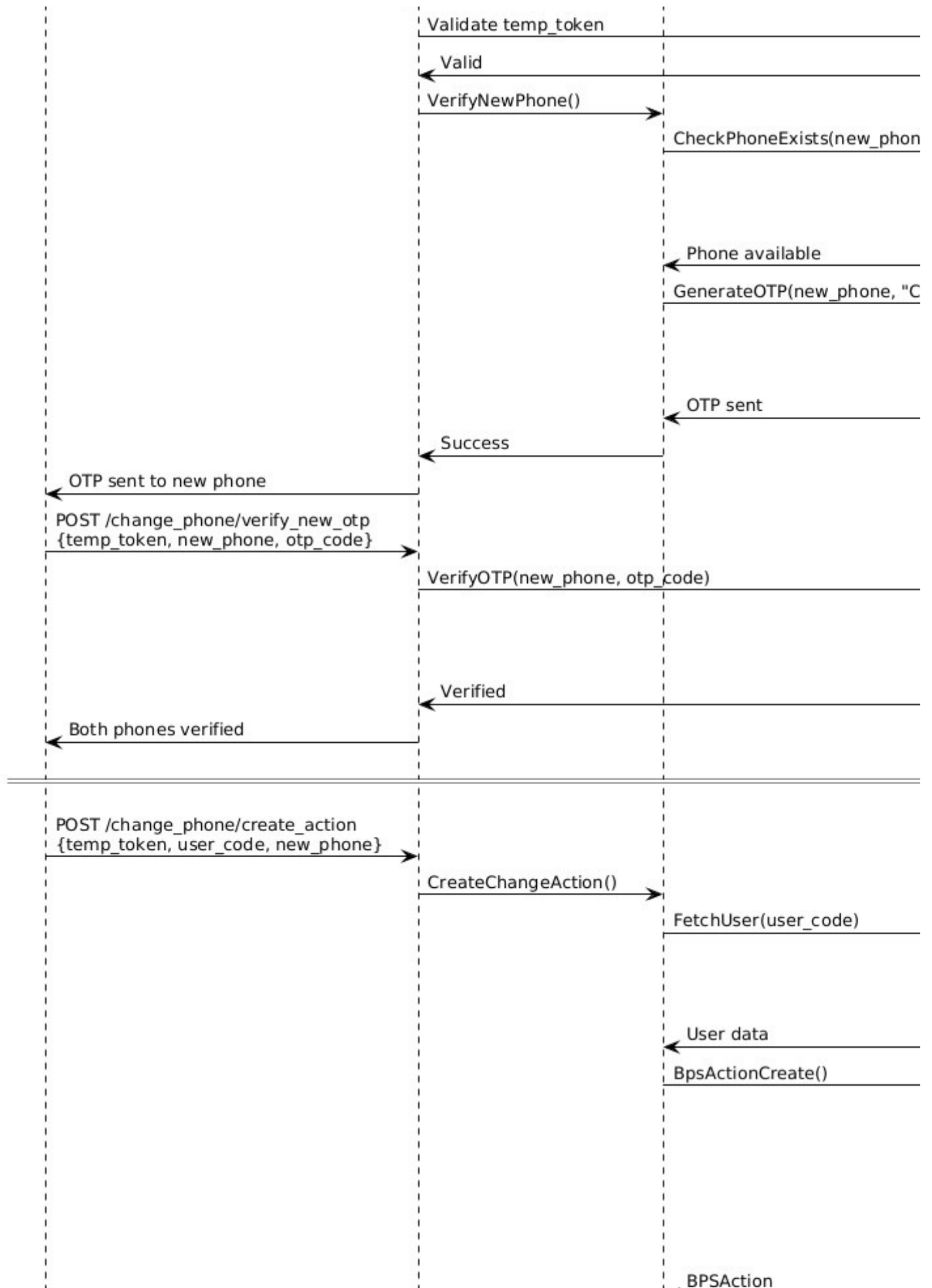
1. Verify current phone number with OTP
2. Verify new phone number with OTP
3. New phone number must not be registered to another user

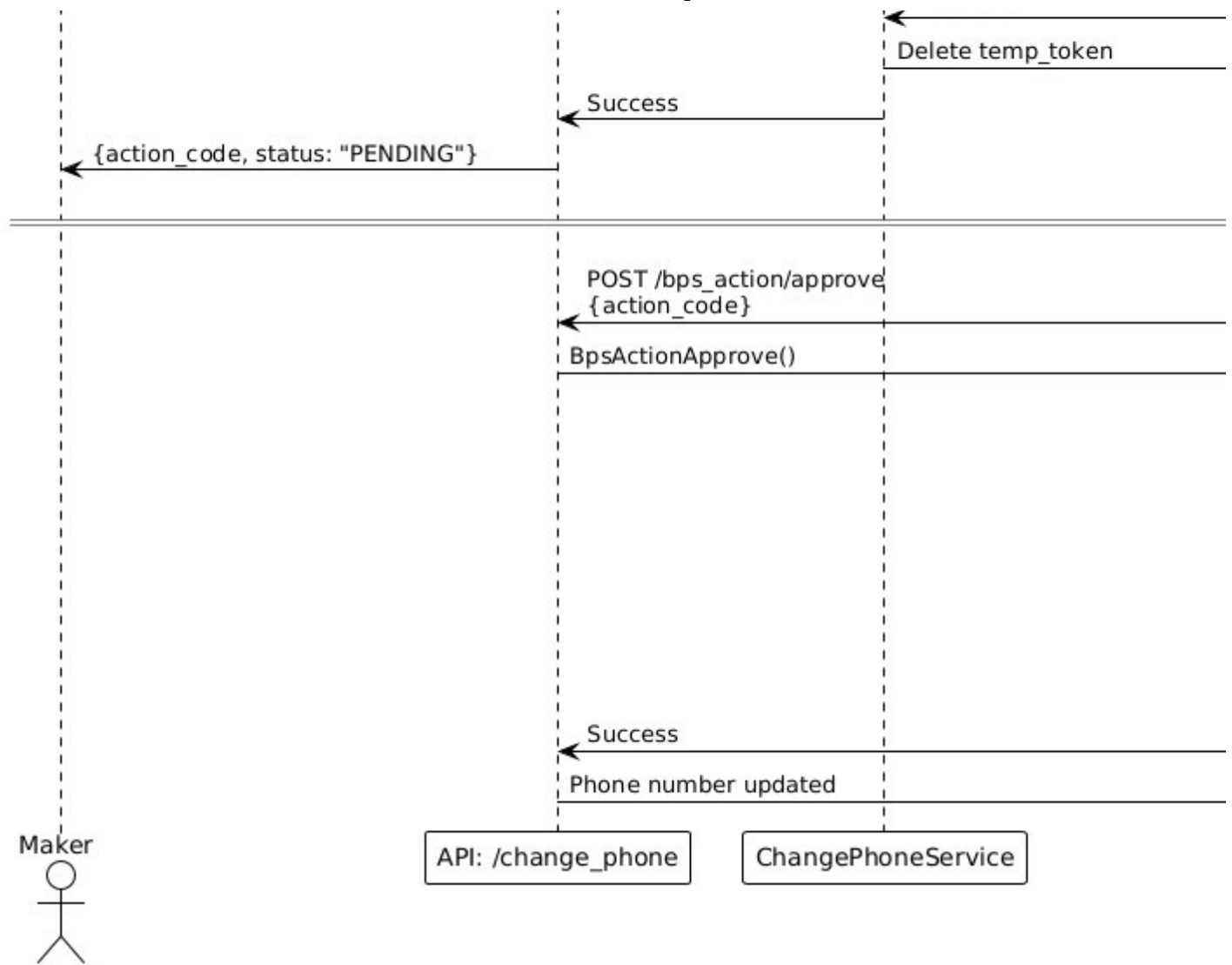
4. Temporary token issued between OTP verifications

5. Maker-Checker approval required

Architectural Flow:







Implementation Highlights:

Temporary Token Management:

```

func (s *Service) GenerateTempToken(ctx context.Context, userCode string) (string,
error) {
    token := uuid.New().String()
    key := fmt.Sprintf("change_phone:%s", token)

    // Store in Redis with 10-minute expiration
    err := s.redis.Set(ctx, key, userCode, 10*time.Minute).Err()
    if err != nil {
        return "", err
    }

    return token, nil
}
  
```

```

func (s *Service) ValidateTempToken(ctx context.Context, token string) (string,
  
```

```
error) {  
    key := fmt.Sprintf("change_phone:%s", token)  
    userCode, err := s.redis.Get(ctx, key).Result()  
    if err == redis.Nil {  
        return "", errors.NewAppError("BPS-401", "Invalid or expired token")  
    }  
    return userCode, err  
}
```

5.4 UPDATE_TRANSFER_LIMIT

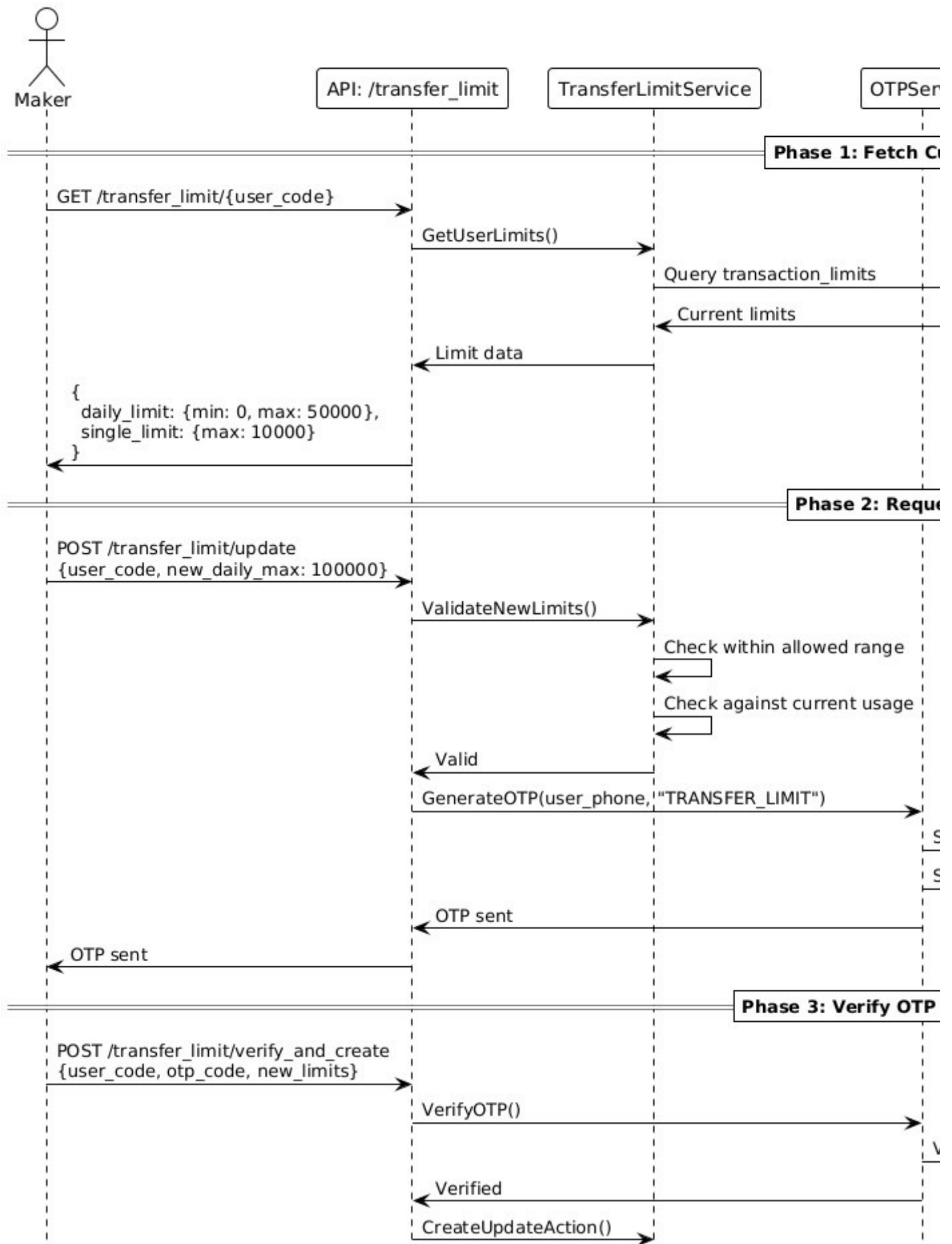
Purpose: Modify transaction limits for a user's account.

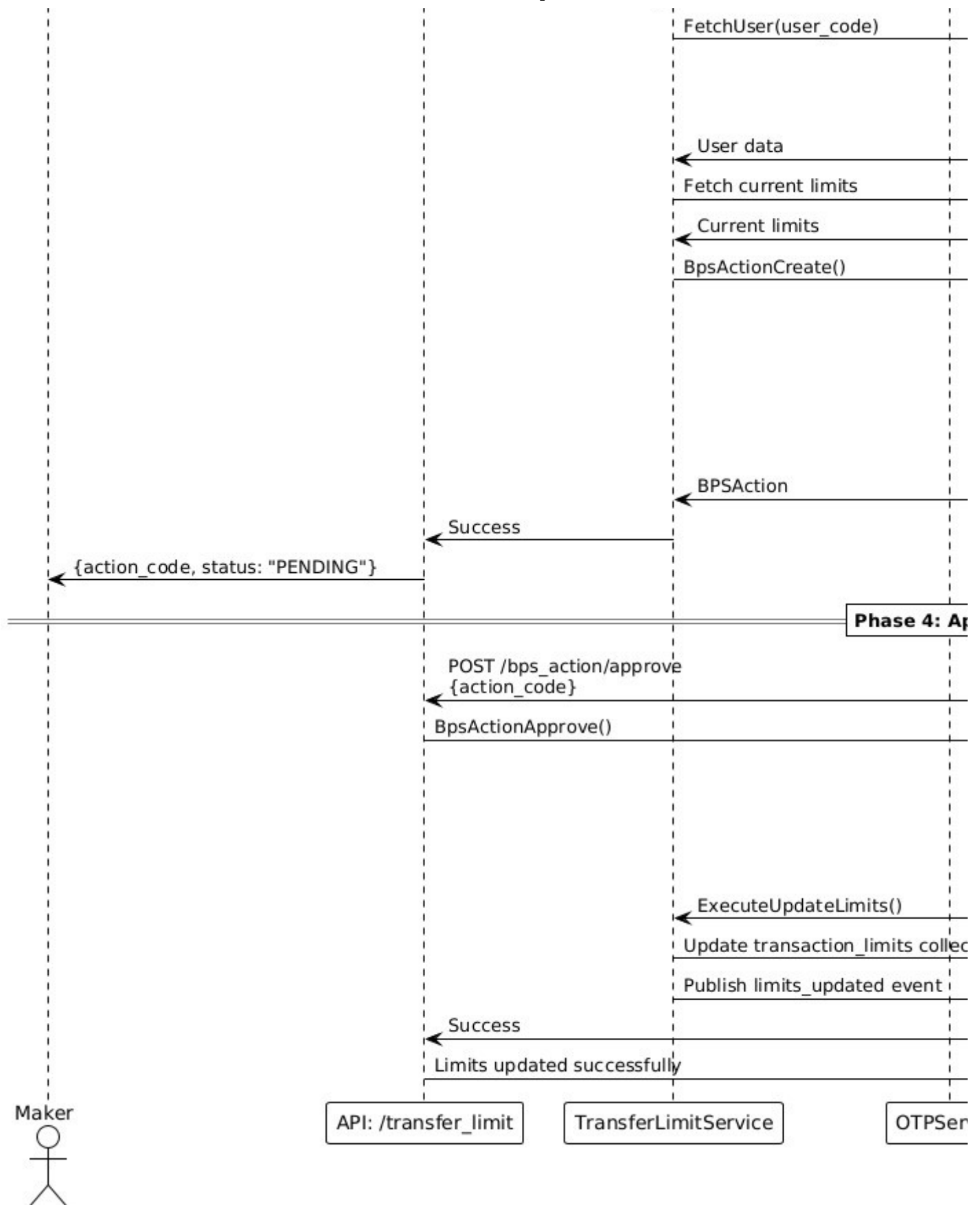
Business Rules:

1. OTP verification required
2. New limits must be within allowed ranges (min/max)
3. Cannot decrease limits below current day's usage
4. Separate limits for daily and single transaction
5. Different limits per service (e.g., transfer, bill payment)

Architectural Flow:

Update Transfer Lim





Limit Validation Logic:

```
func (s *Service) ValidateNewLimits(ctx context.Context, userCode string, newLimits
dto.TransferLimitUpdate) error {
```



```

// Fetch current limits
current, err := s.storage.GetUserLimits(ctx, userCode)
if err != nil {
    return err
}

// Validate daily limit
if newLimits.DailyMax < s.cfg.MinDailyLimit || newLimits.DailyMax >
s.cfg.MaxDailyLimit {
    return errors.NewAppError("BPS-400", "Daily limit out of allowed range")
}

// Validate single transaction limit
if newLimits.SingleMax > newLimits.DailyMax {
    return errors.NewAppError("BPS-400", "Single limit cannot exceed daily
limit")
}

// Check current day usage
usage, _ := s.GetTodayUsage(ctx, userCode)
if newLimits.DailyMax < usage {
    return errors.NewAppError("BPS-400", "Cannot set limit below today's
usage")
}

return nil
}

```

5.5 ACCESS_CONTROL

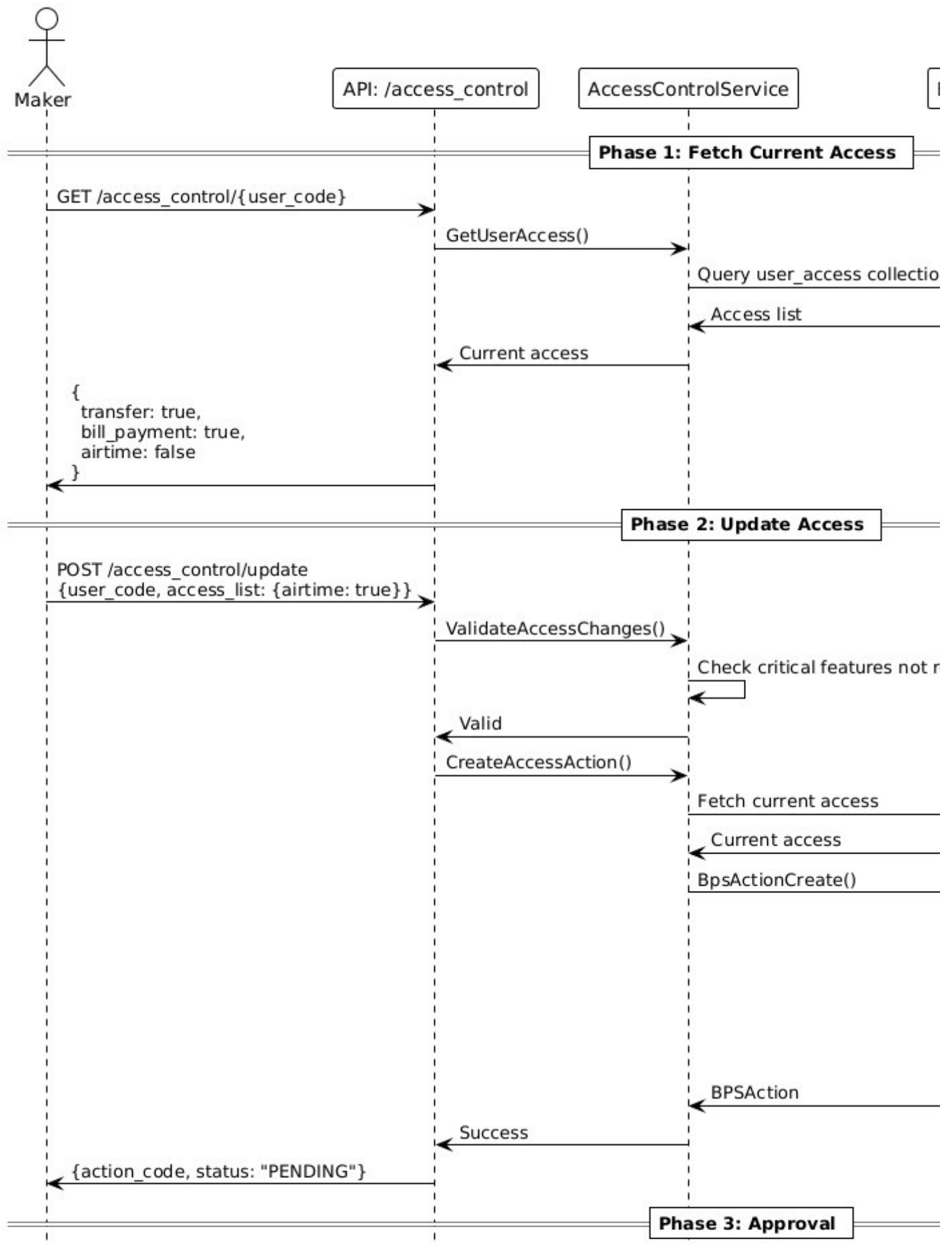
Purpose: Manage user permissions and feature access.

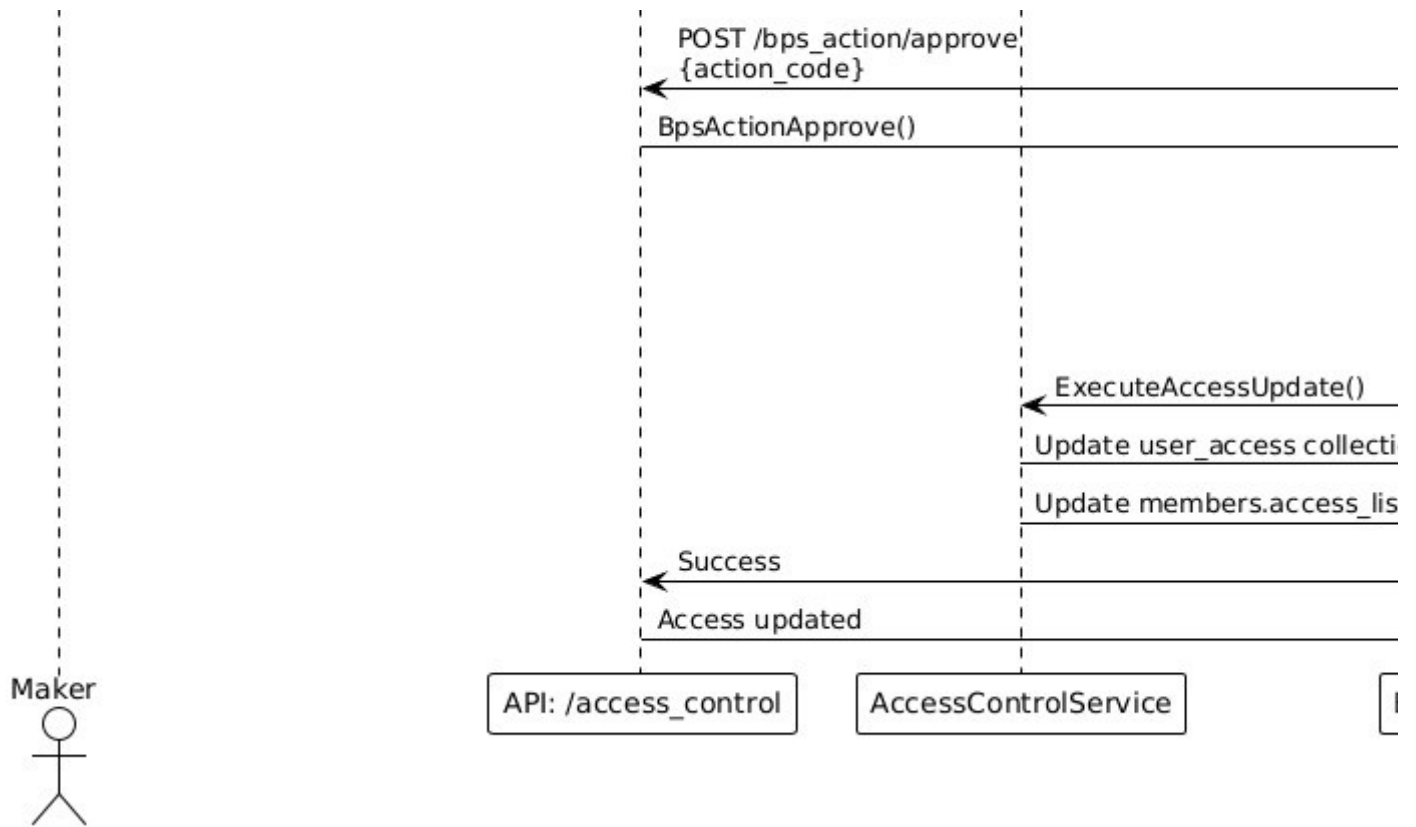
Business Rules:

1. Access list defines which features user can access
2. Changes tracked (previous vs current access)
3. Cannot remove critical access (e.g., view balance)
4. Maker-Checker approval required

Architectural Flow:

Access Control - Complete I





5.6 ENABLE_DISABLE_USER

Purpose: Enable or disable user account access.

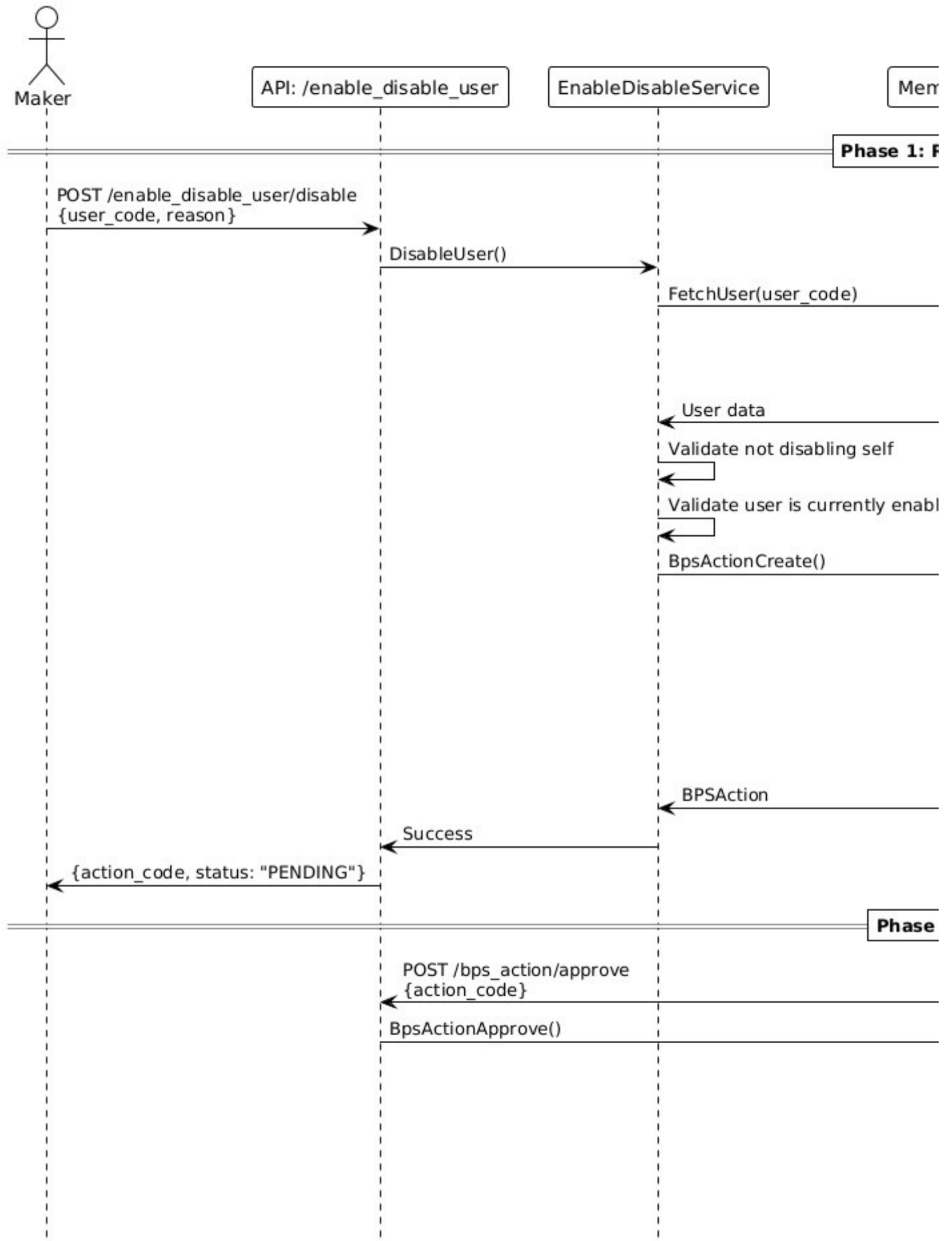
Business Rules:

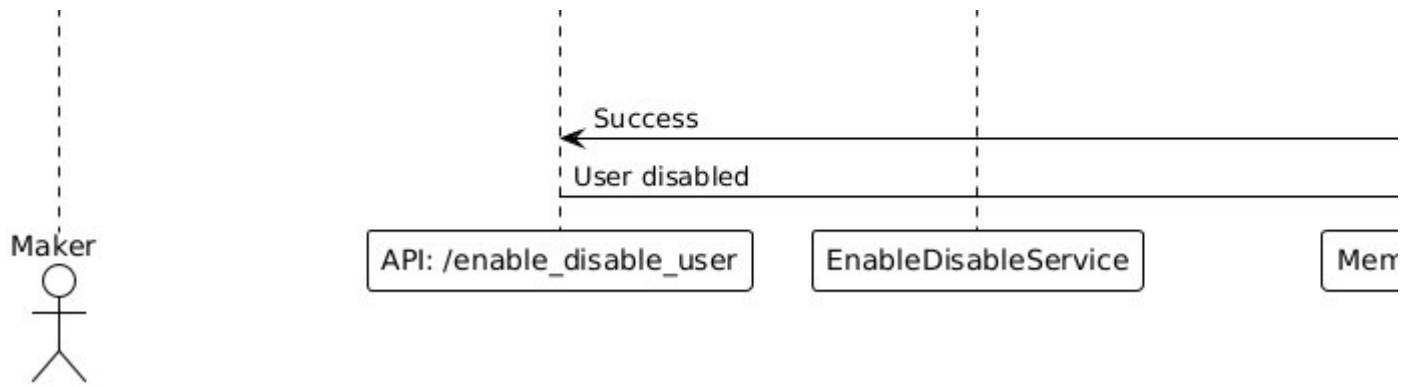
1. Cannot disable own account
2. Disabled users cannot login
3. Reason required for disabling
4. Maker-Checker approval required

Architectural Flow:



Enable/Disable U





5.7 CHEQUE_AUTHORIZATION

Purpose: Authorize cheque payments.

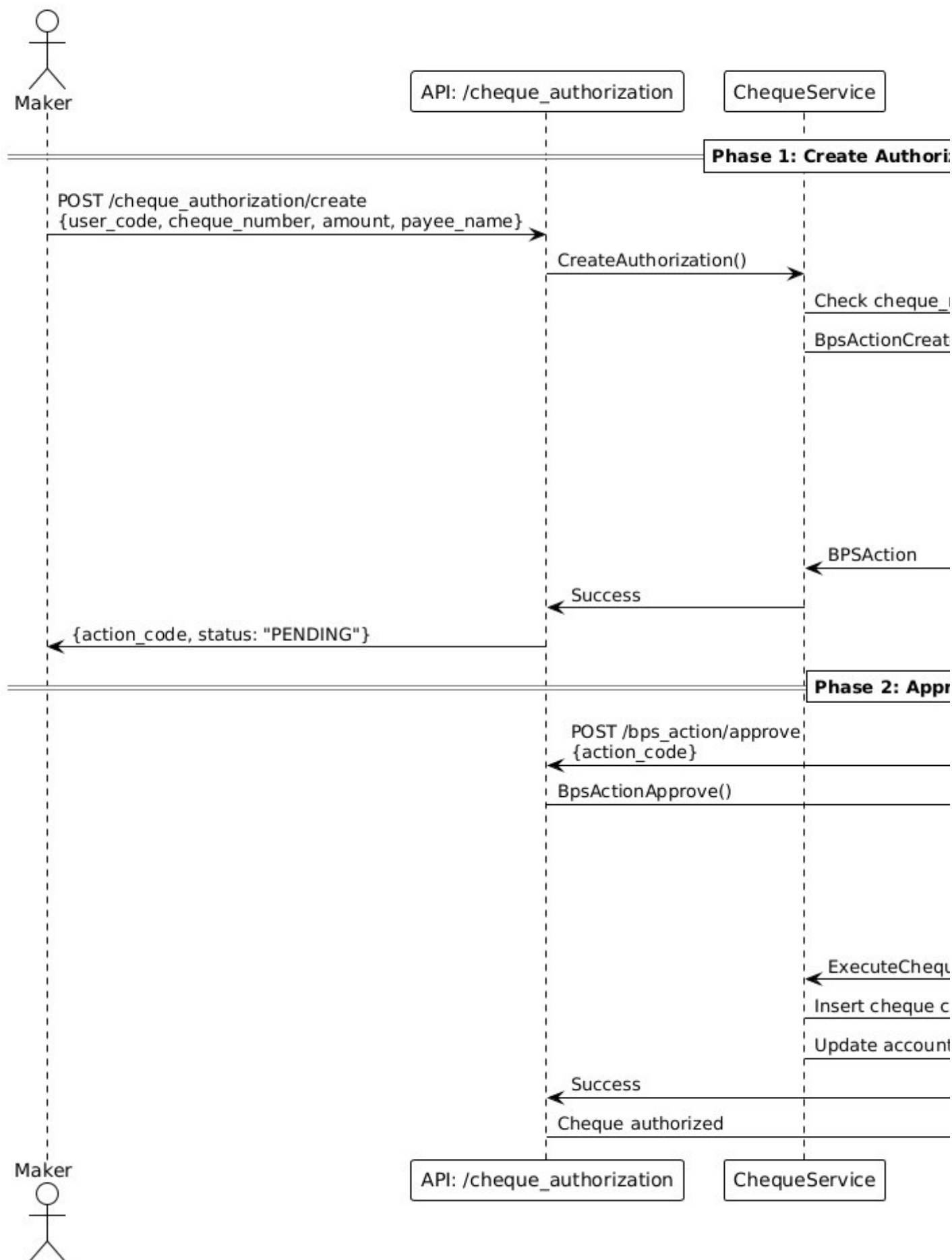
Business Rules:

1. Cheque number
2. Payee information required
3. Maker-Checker approval required

Architectural Flow:



Cheque Authorization



API Endpoints:

- POST /attach_phone/validate
- POST /attach_phone/send_otp
- POST /attach_phone/verify_otp
- POST /attach_phone/attach

5.9 DETACH_PHONE_NUMBER

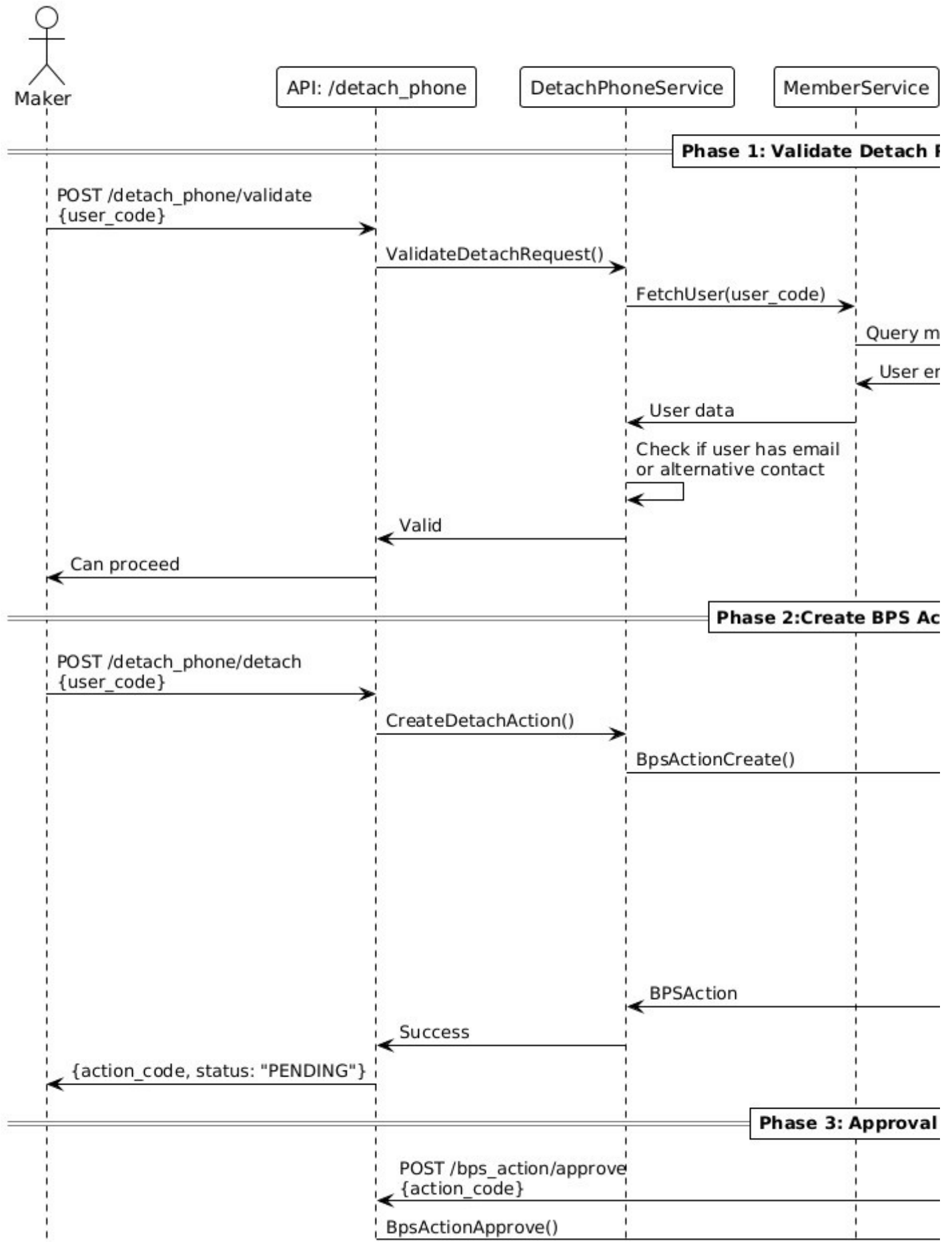
Purpose: Remove a phone number from a user account.

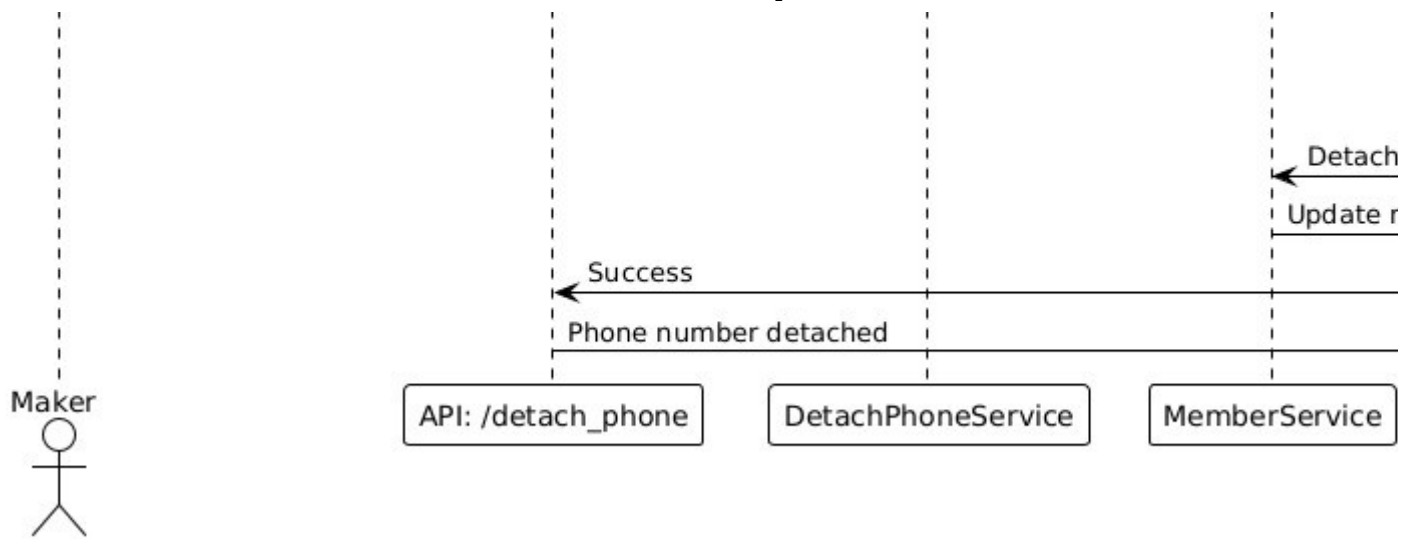
Business Rules:

1. Cannot detach if it's the only contact method
2. User must have alternative contact (email)
3. OTP verification optional
4. Maker-Checker approval required

Architectural Flow:

Detach Phone Number - (





5.10 UNLINK_DEVICE

Purpose: Unlink a device from user's account (when device lost/stolen).

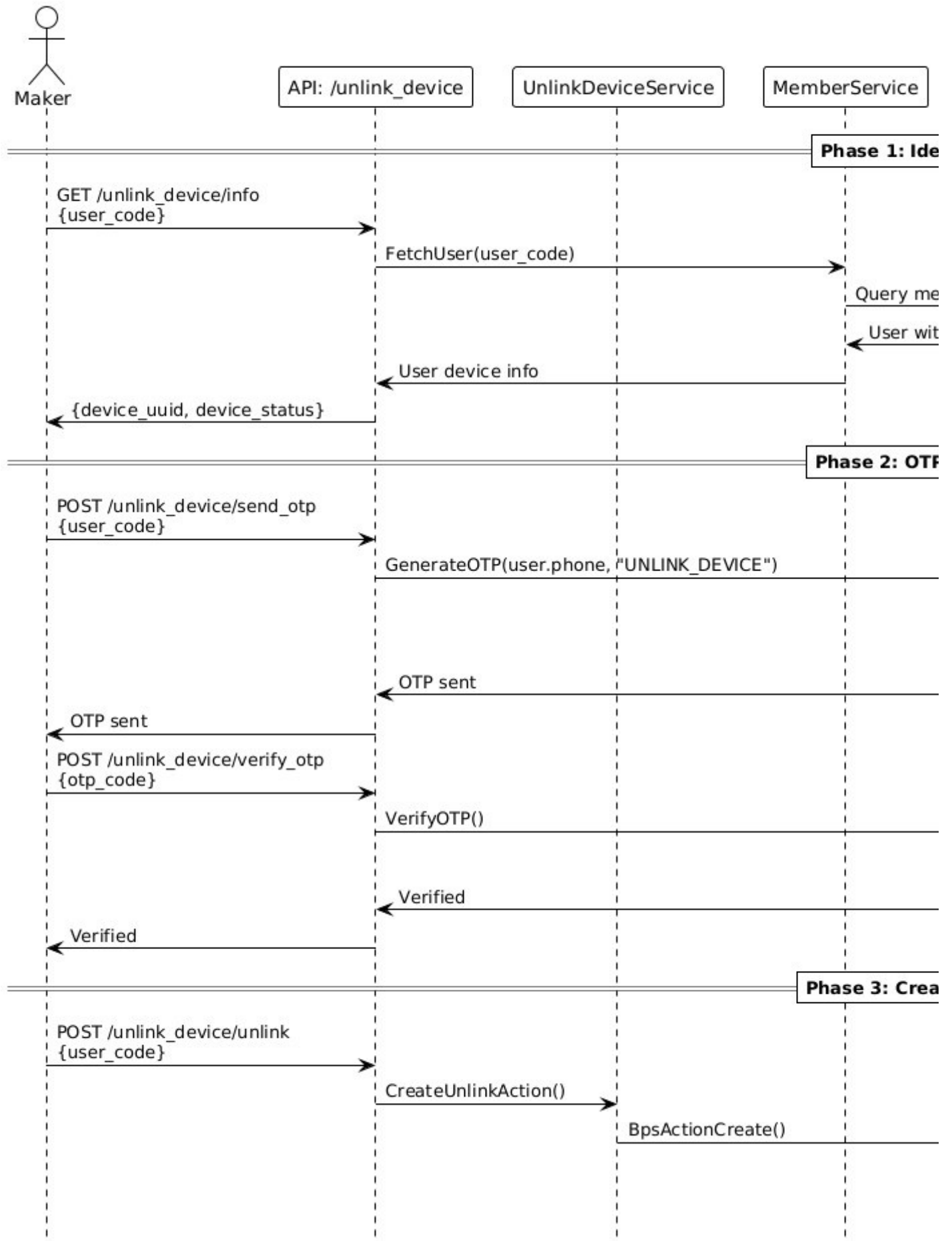
Business Rules:

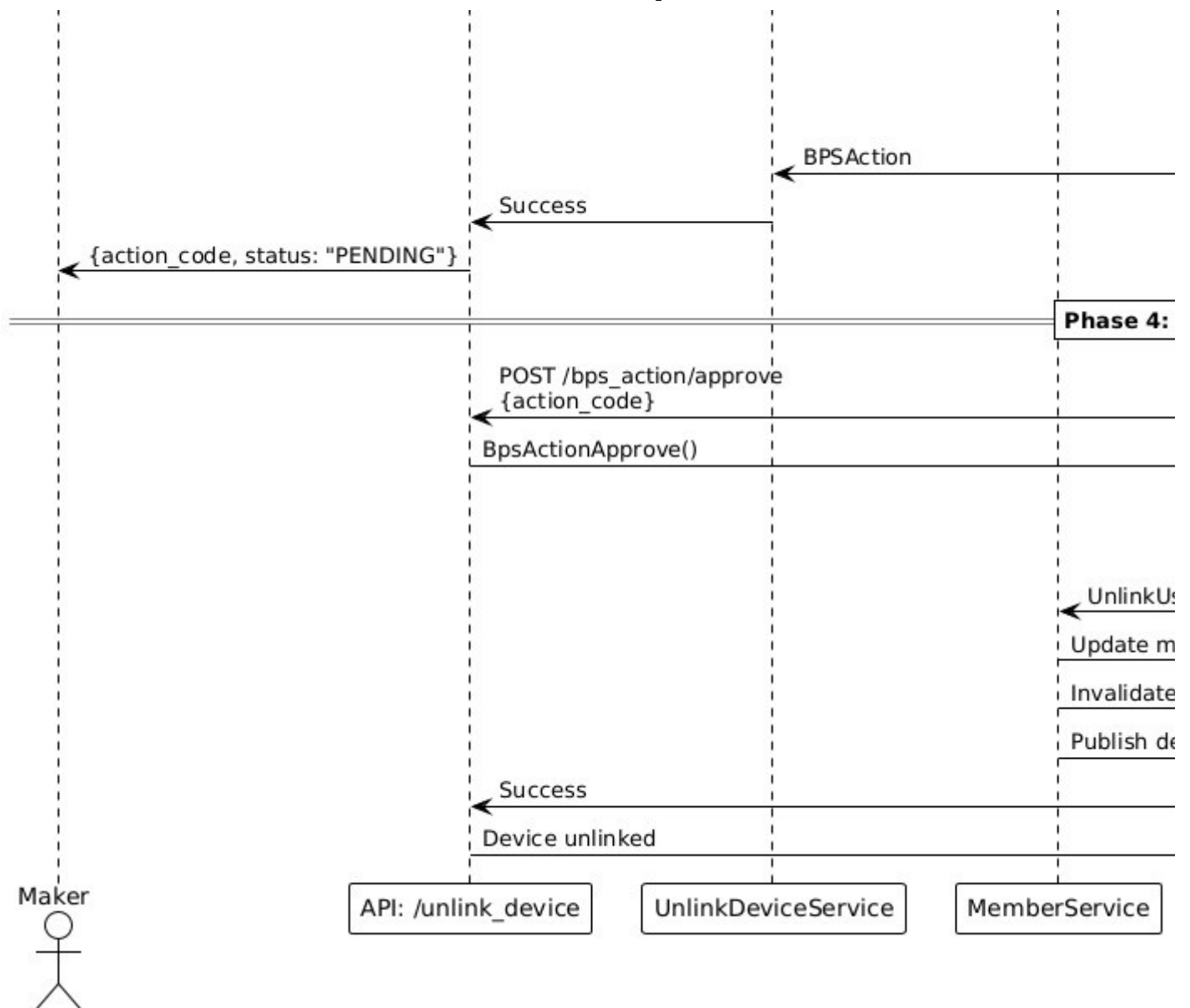
1. Device identified by UUID
2. Unlinking invalidates all device sessions
3. OTP verification required
4. User notified via SMS
5. Maker-Checker approval required

Architectural Flow:



Unlink Device -





5.11 ACTIVATE_ACCOUNT

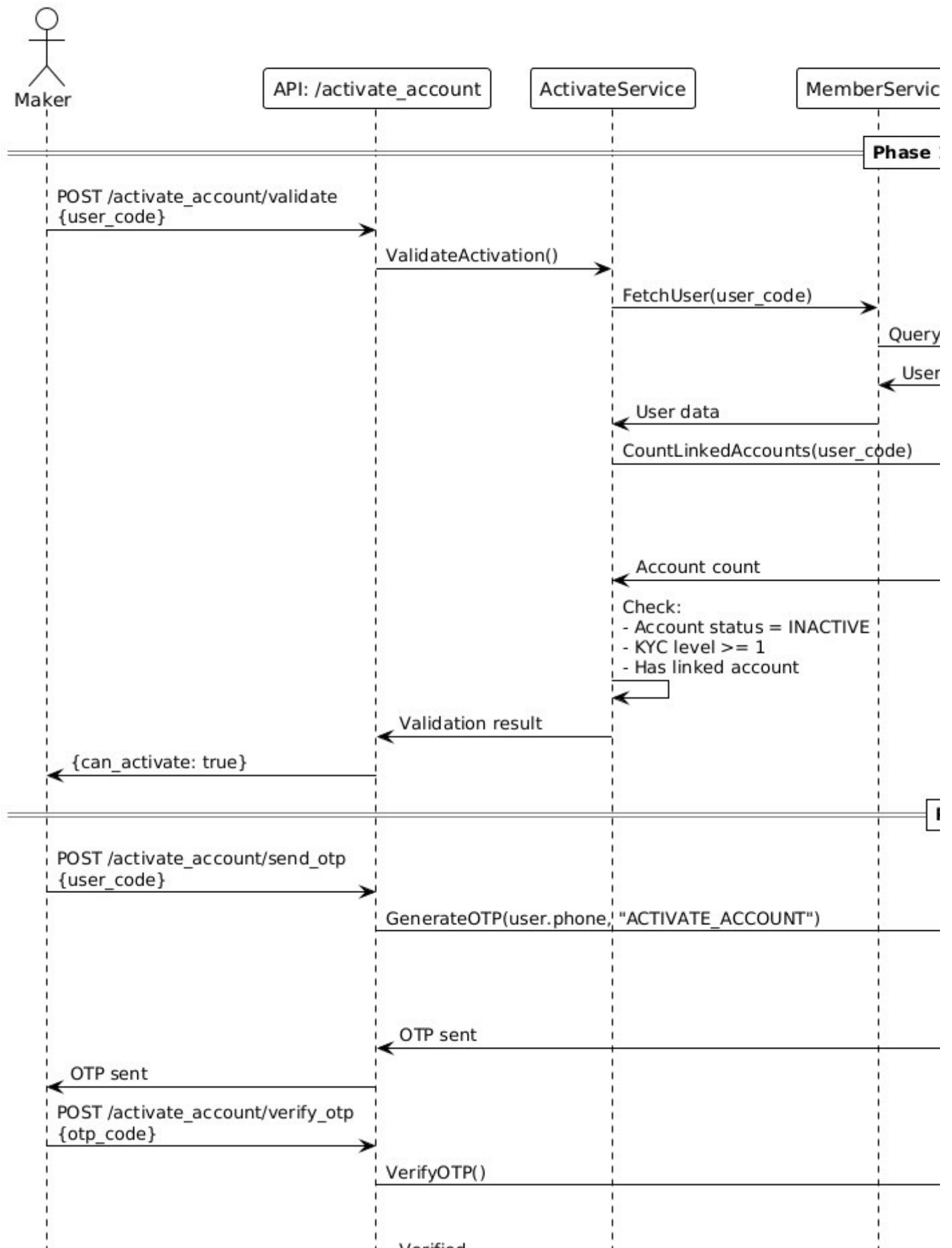
Purpose: Activate a newly registered or inactive user account.

Business Rules:

1. Account must be in INACTIVE or PENDING status
2. KYC verification required (minimum Level 1)
3. At least one linked account required
4. OTP verification required
5. Maker-Checker approval required

Architectural Flow:

Activate /





5.12 UNLINK_ACCOUNT

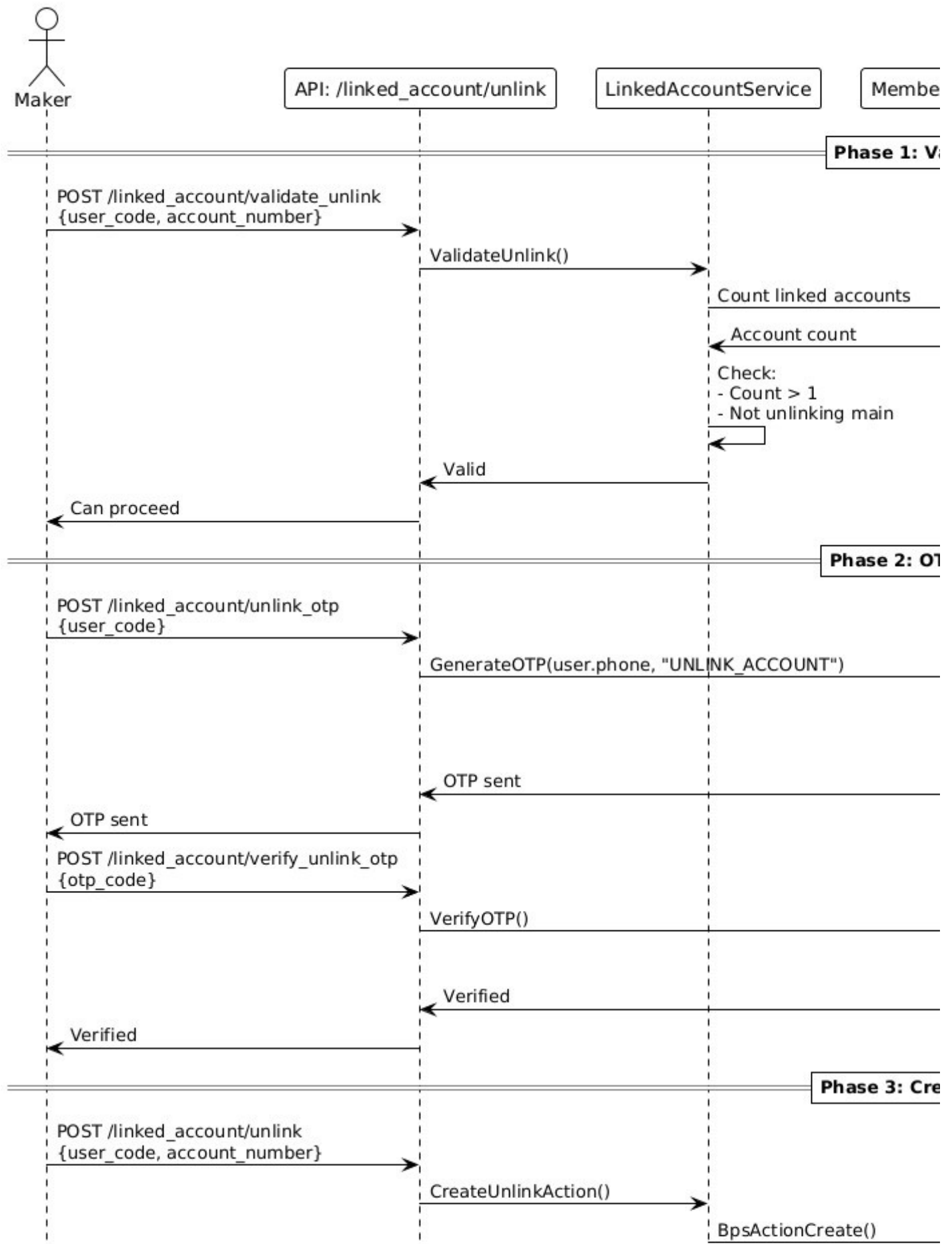
Purpose: Unlink a bank account from user's mobile app profile.

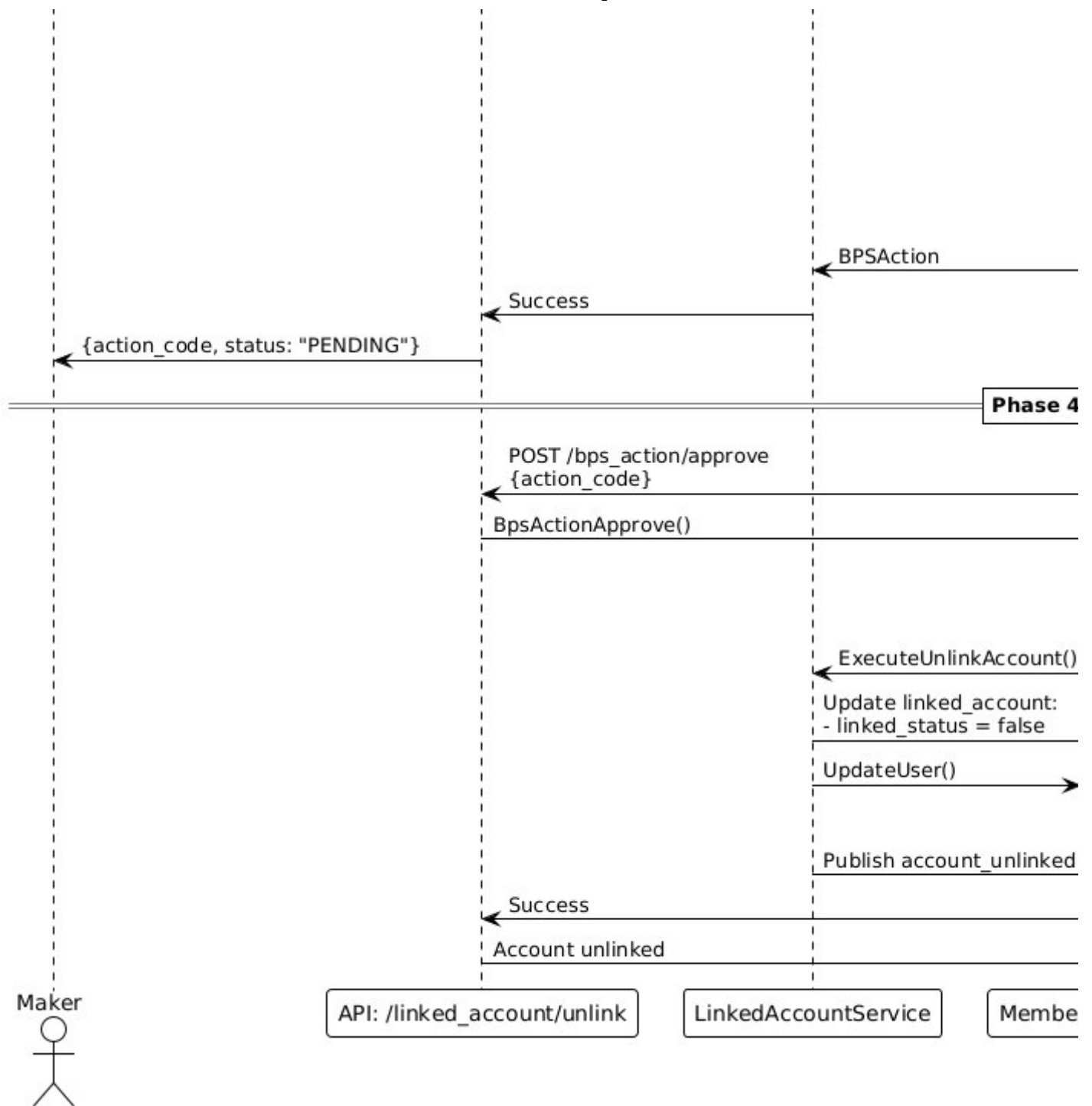
Business Rules:

1. Cannot unlink if it's the only account
2. Cannot unlink main account without designating new main
3. OTP verification required
4. Maker-Checker approval required

Architectural Flow:

Unlink Account





5.13 UPGRADE_KYC_LEVEL

Purpose: Upgrade user's KYC verification level.

Business Rules:

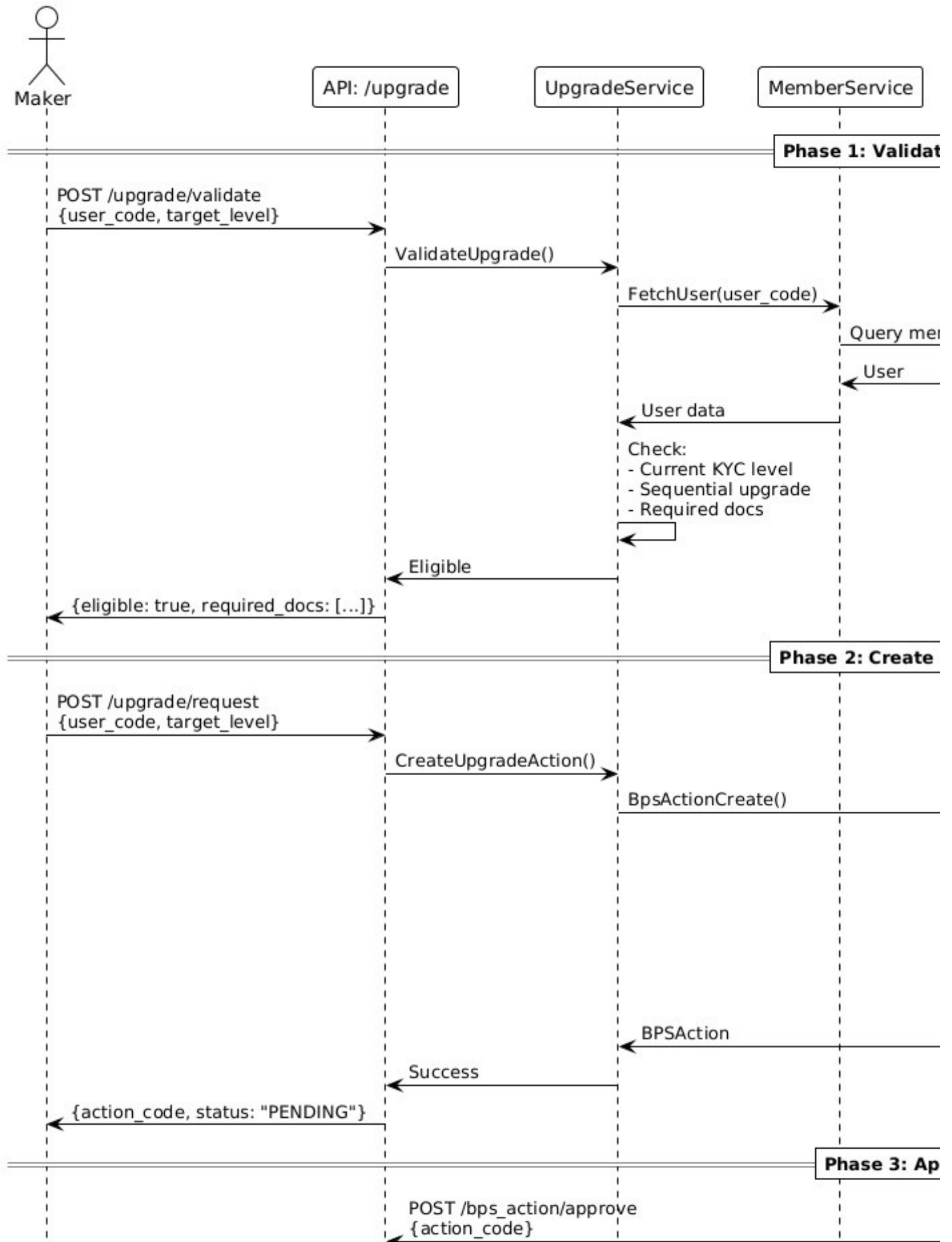
1. KYC levels: 0 (unverified), 1 (basic), 2 (enhanced)
2. Cannot skip levels
3. Each level requires documents

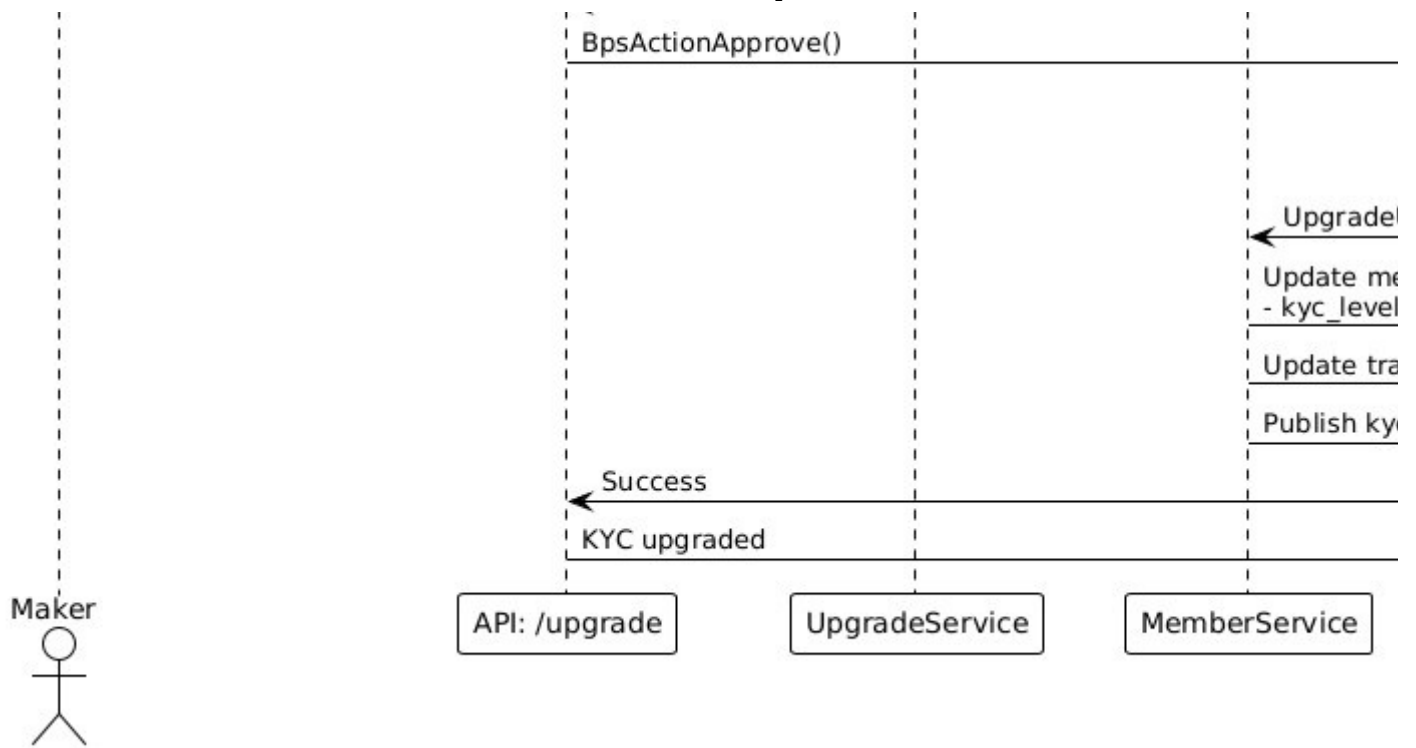
4. Higher levels = higher transaction limits

5. Maker-Checker approval required

Architectural Flow:

Upgrade KYC Level





5.14 CREATE_TRANSFER_LIMIT

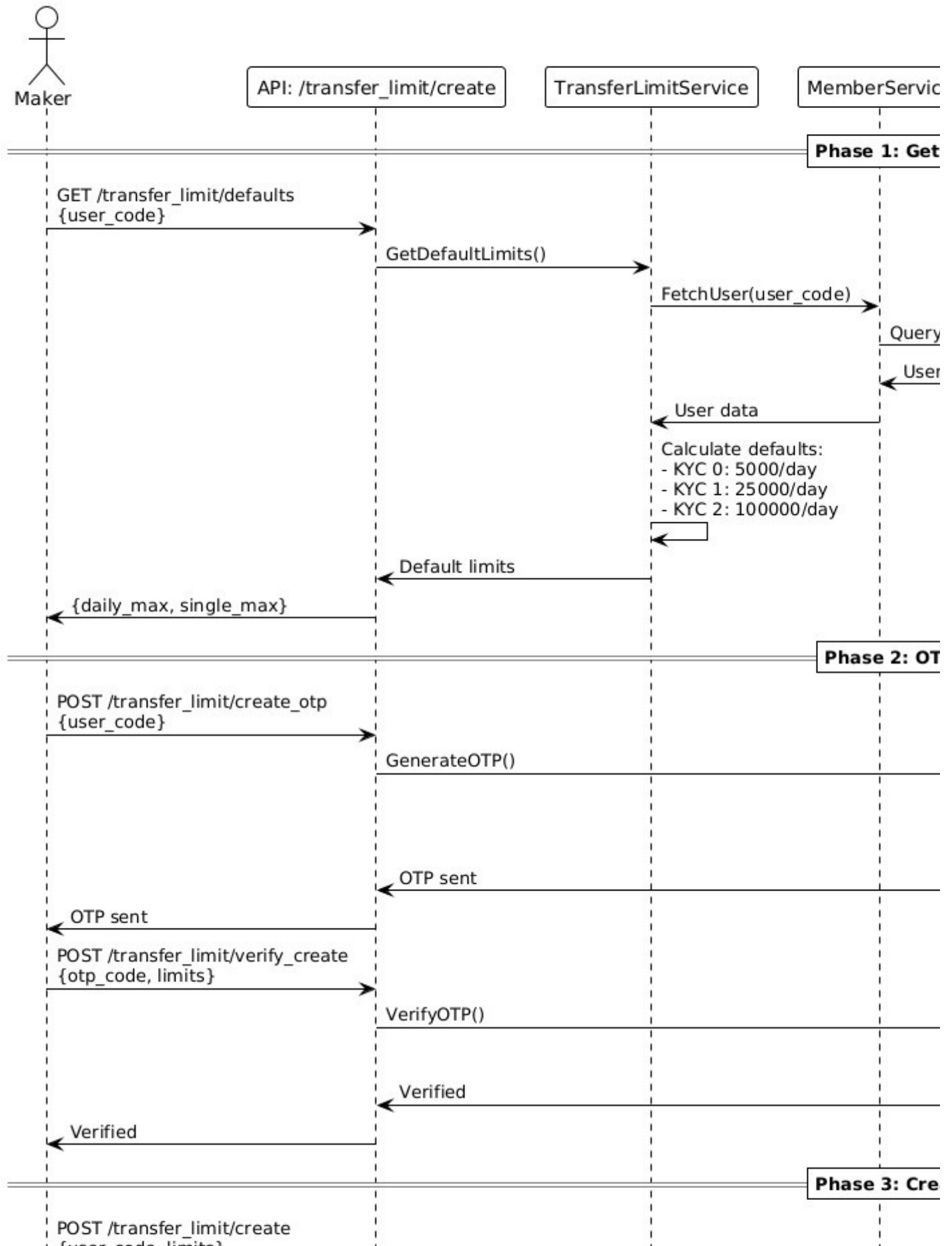
Purpose: Create initial transaction limits for a user.

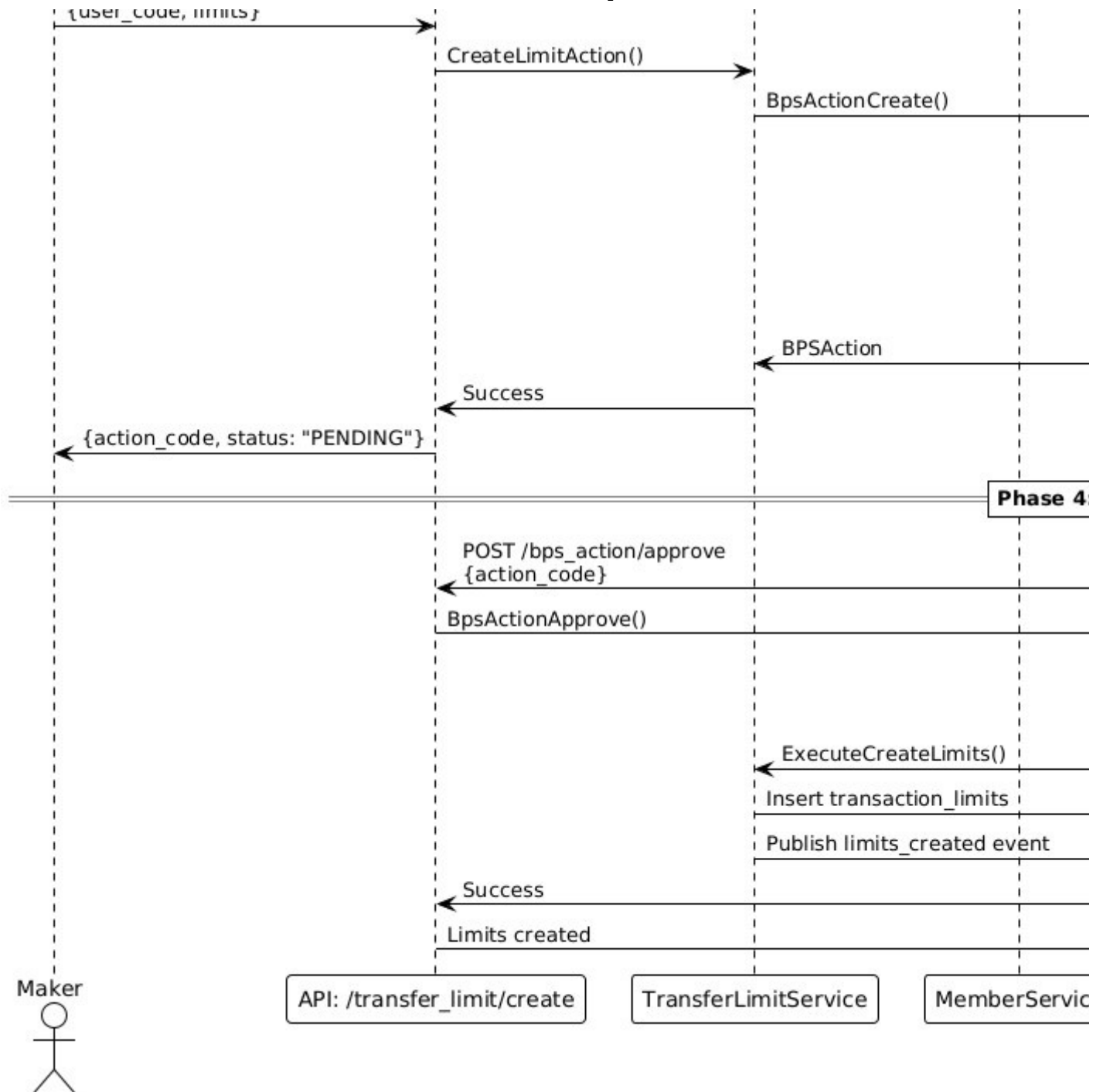
Business Rules:

1. Default limits based on KYC level
2. Can customize within allowed ranges
3. OTP verification required
4. Maker-Checker approval required

Architectural Flow:

Create Transfer Li





5.15 RESET_TRANSFER_LIMIT

Purpose: Reset transaction limits to default values.

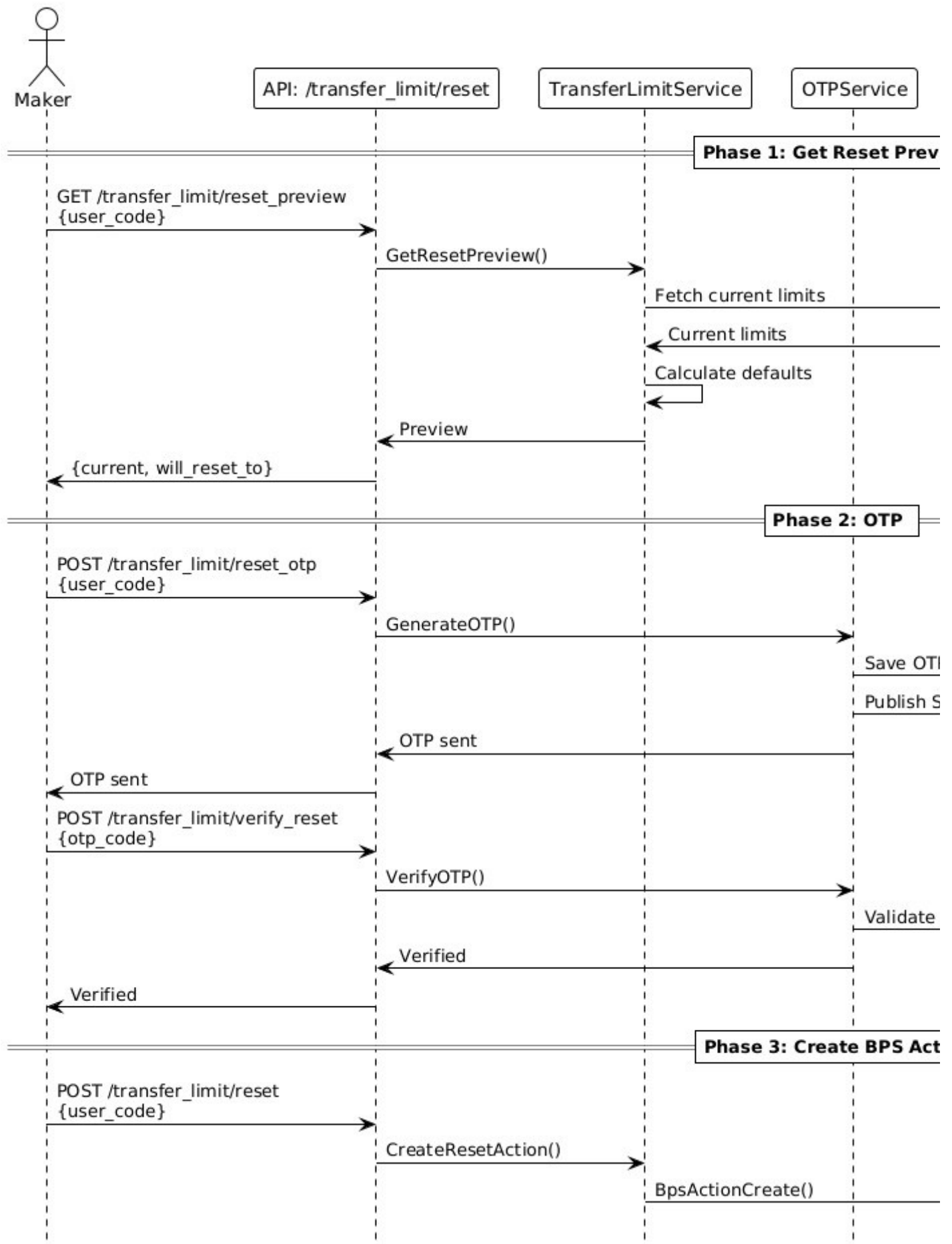
Business Rules:

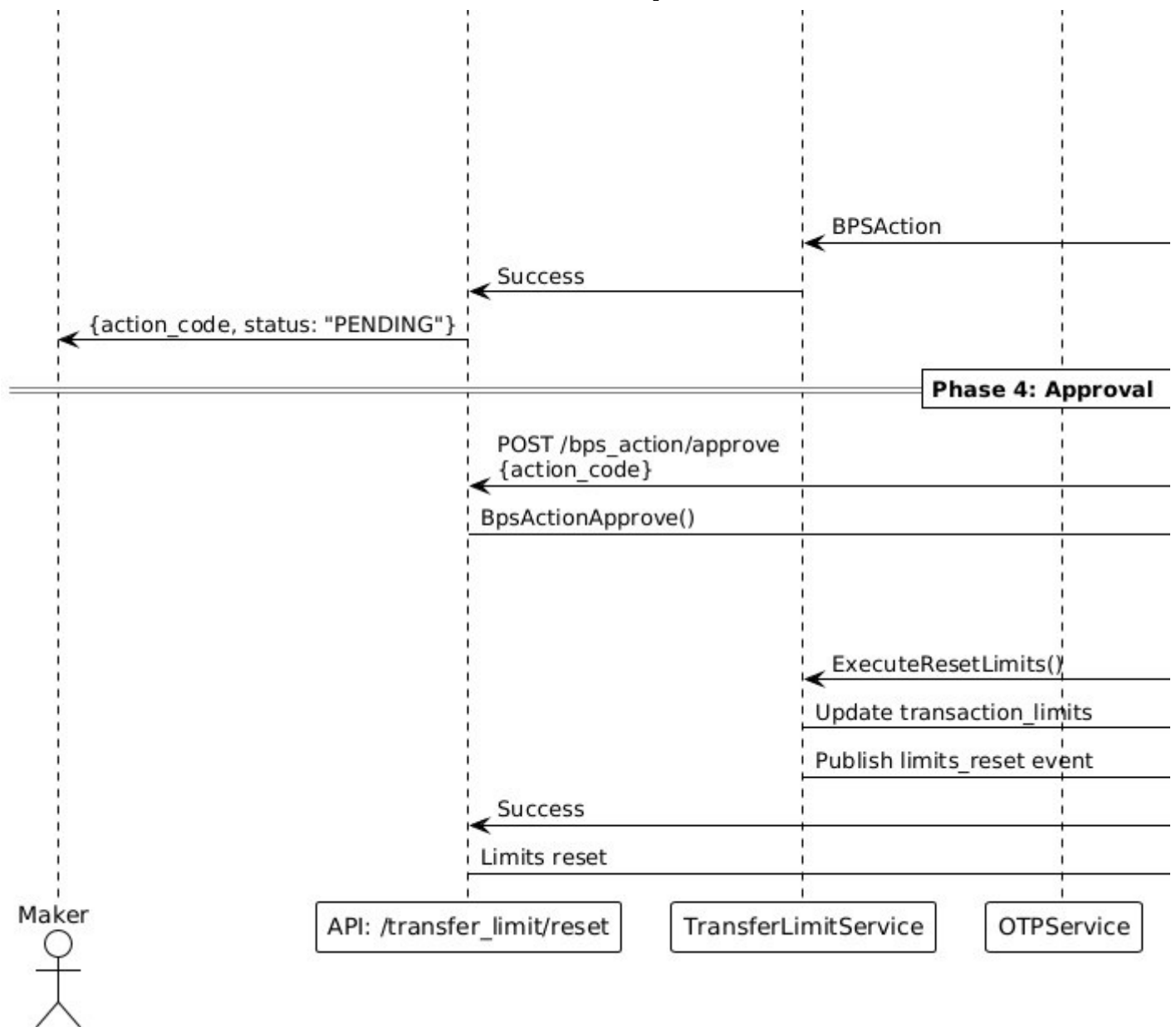
1. Resets to defaults based on KYC level
2. OTP verification required
3. Cannot reset below current day's usage

4. Maker-Checker approval required

Architectural Flow:

Reset Transfer Limit - Co





5.17 CHANGE_EMAIL

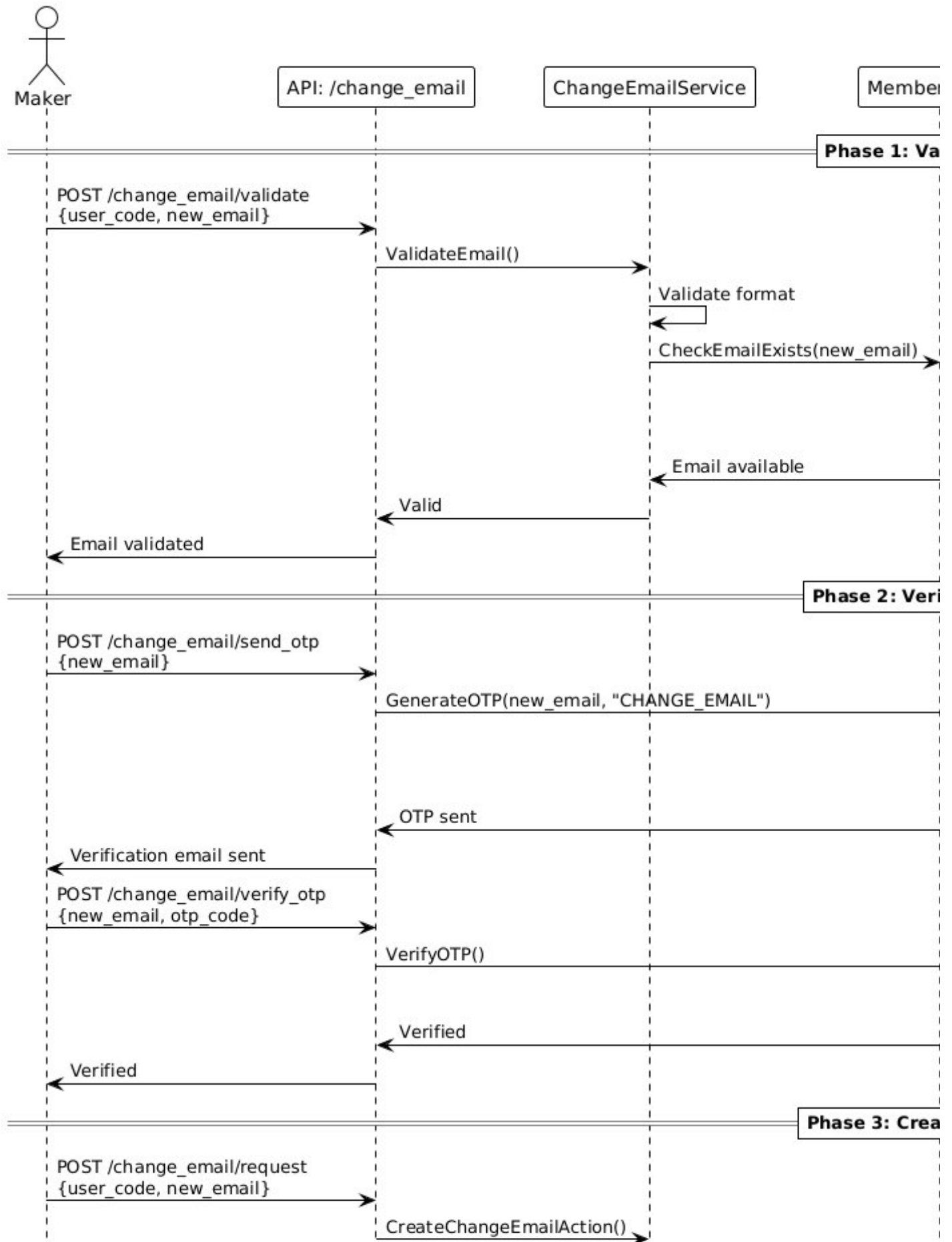
Purpose: Update a user's registered email address.

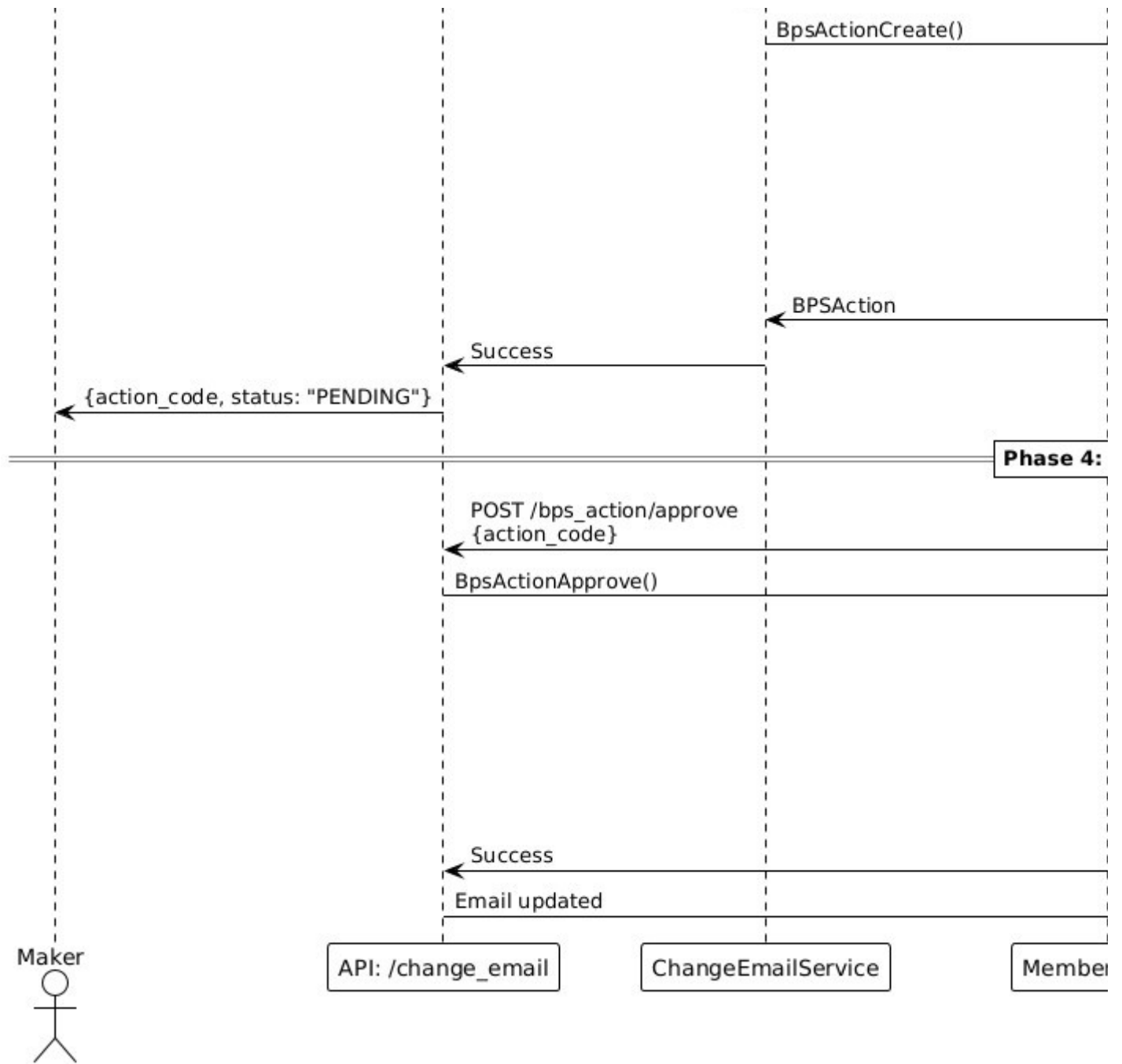
Business Rules:

1. Email must be valid format
2. Not registered to another user
3. OTP verification to new email required
4. Maker-Checker approval required

Architectural Flow:

Change Email -





5.18 Action Summary Table

Action Code	OTP Required	Approval Required	Database Collections Updated
LINK_ACCOUNT	✓	✓	bps_actions, linked_account, members

UNLINK_ACCOUNT	✓	✓	bps_actions, linked_account, members
CHANGE_PHONE_NUMBER	✓ (dual)	✓	bps_actions, members
UPDATE_TRANSFER_LIMIT	✓	✓	bps_actions, transaction_limits
ACCESS_CONTROL	✗	✓	bps_actions, user_access, members
ENABLE_DISABLE_USER	✗	✓	bps_actions, members
CHEQUE_AUTHORIZATION	✗	✓	bps_actions, cheque
ATTACH_PHONE_NUMBER	✓	✓	bps_actions, members
DETACH_PHONE_NUMBER	Optional	✓	bps_actions, members
UNLINK_DEVICE	✓	✓	bps_actions, members
ACTIVATE_ACCOUNT	✓	✓	bps_actions, members
UPGRADE_KYC_LEVEL	✗	✓	bps_actions, members, transaction_limits
CREATE_TRANSFER_LIMIT	✓	✓	bps_actions, transaction_limits
RESET_TRANSFER_LIMIT	✓	✓	bps_actions, transaction_limits
CHANGE EMAIL	✓	✓	bps actions. members